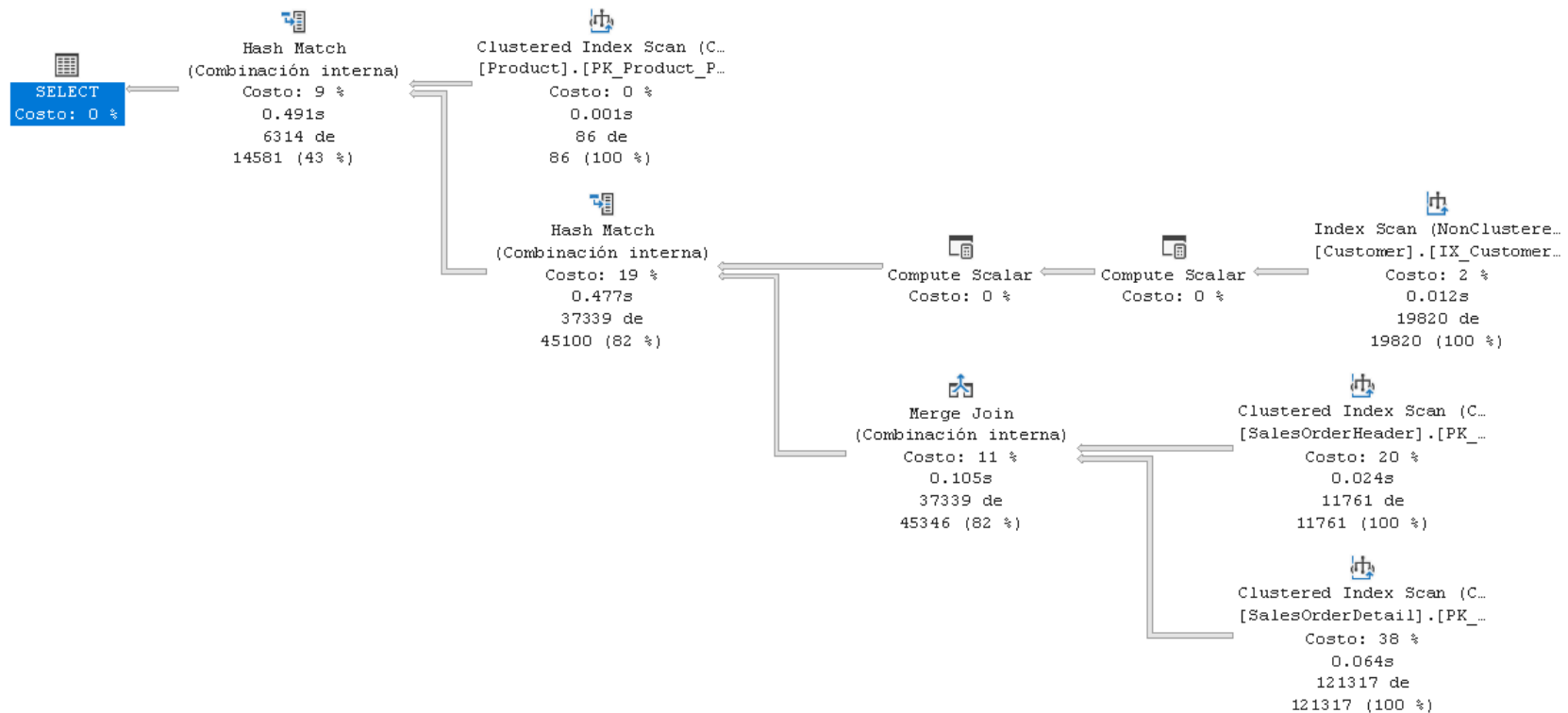


Base de datos Distribuidas – Práctica 2

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	19-03-26	668 ms	547 ms	SOD=1238 SOH=686 Customer=37 Product=15	Clustered index scan 38%	-El where en la consulta genera un alto costo. -Aparecen Clustered Index Scan tanto en SOD y SOH	-Reescribir el where -Implementar un índice con include en sod -Implementar un índice con include en soh

Grafico Plan Real Original

Base de datos Distribuidas – Práctica 2



Estadísticas originales

(6314 filas afectadas)

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderDetail'.

Scan count 1,
logical reads 1238,

```
physical reads 3, page server reads 0,  
read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'SalesOrderHeader'.  
Scan count 1,  
logical reads 686,  
physical reads 3, page server reads 0,  
read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'Customer'.  
Scan count 1,  
logical reads 37,  
physical reads 1, page server reads 0,  
read-ahead reads 35, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'Product'.  
Scan count 1,  
logical reads 15,  
physical reads 1, page server reads 0,  
read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
  
(1 fila afectada)  
  
SQL Server Execution Times:  
CPU time = 547 ms, elapsed time = 668 ms.
```

Después de corregir parte del where (solo se reducen los tiempos).

Statistics después de corregir el where:

```
Table 'SalesOrderDetail'.  
Scan count 1,  
logical reads 1238,  
physical reads 3, page server reads 0,  
read-ahead reads 1234, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'SalesOrderHeader'.
```

```
Scan count 1,  
logical reads 686,  
physical reads 3, page server reads 0,  
read-ahead reads 682, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'Customer'.  
Scan count 1,  
logical reads 37,  
physical reads 1, page server reads 0,  
read-ahead reads 35, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
Table 'Product'.  
Scan count 1,  
logical reads 15,  
physical reads 1, page server reads 0,  
read-ahead reads 13, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server  
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.  
  
(1 fila afectada)  
  
SQL Server Execution Times:  
CPU time = 78 ms, elapsed time = 155 ms.
```

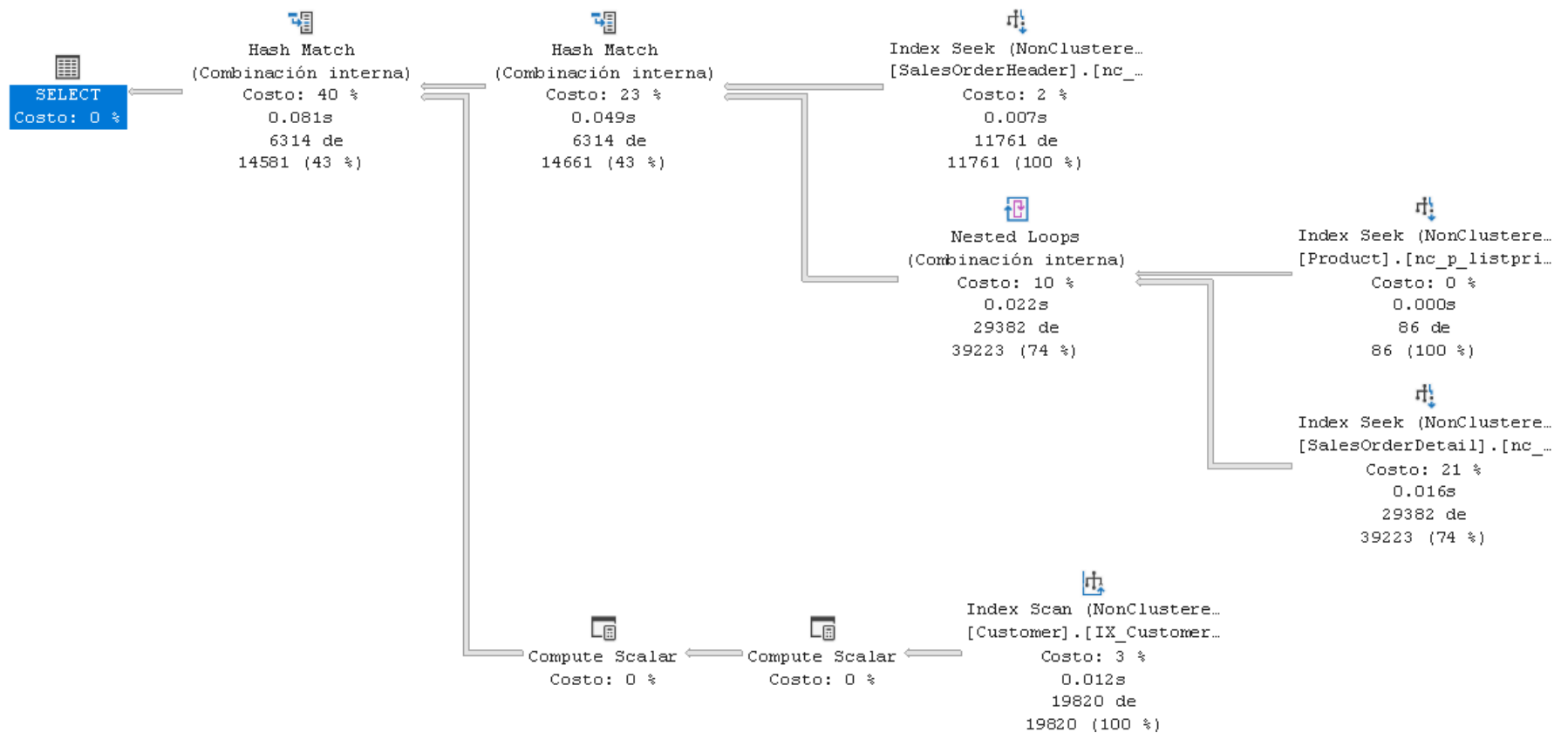
Agregamos índice con include de la tabla del join para Índice de fecha

--creacion de indice nonclustered para Indice de precio

Después de dos índices correctos en salesorderheader y productionproduct

--creamos un nuevo indice para desaparecer el index seek sobre salesorderdetail

--ejecutamos de nuevo con 3 índices optimizados y correctos



Statistics con 3 indices

(6314 filas afectadas)
 Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

```
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page
server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead
reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderDetail'.
Scan count 1,
logical reads 1238,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Customer'.
Scan count 1,
logical reads 37,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderHeader'.
Scan count 1,
logical reads 35,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Product'.
Scan count 1,
logical reads 4,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
    CPU time = 171 ms,  elapsed time = 213 ms.
```

Tabla comparativa:

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
1	19-03-26	668 ms	547 ms	SOD=1238 SOH=686 Customer=37 Product=15	Clustered index scan 38%	-El where en la consulta genera un alto costo. -Aparecen Clustered Index Scan tanto en SOD y SOH	-Reescribir el where -Implementar un índice con include en sod -Implementar un índice con include en soh
1	23-03-26	213 ms	171 ms	SOD=1238 SOH=37 Customer=35 Product=4	Clustered index scan 38%	-Se lograron 3 index seek por medio de la creación de los índices	-Para esta consulta se logro hacerlo funcional y con bajo costo

CONSULTAS:

Original

```

set statistics io on;
set statistics time on;
SELECT p.Name AS Producto, sod.OrderQty,
soh.OrderDate, c.AccountNumber AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID =
sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID =
soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE YEAR(soh.OrderDate) = 2014 AND p.ListPrice >
1000;
set statistics io off;
set statistics time off;

```

Modificado

```

set statistics io on;
set statistics time on;
SELECT p.Name AS Producto, sod.OrderQty, soh.OrderDate,
c.AccountNumber AS Cliente
FROM Production.Product p
JOIN Sales.SalesOrderDetail sod ON p.ProductID =
sod.ProductID
JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID =
soh.SalesOrderID
JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID
WHERE soh.OrderDate >= '2014-01-01'
AND soh.OrderDate < '2015-01-01'
AND p.ListPrice > 1000;
set statistics io off;
set statistics time off;

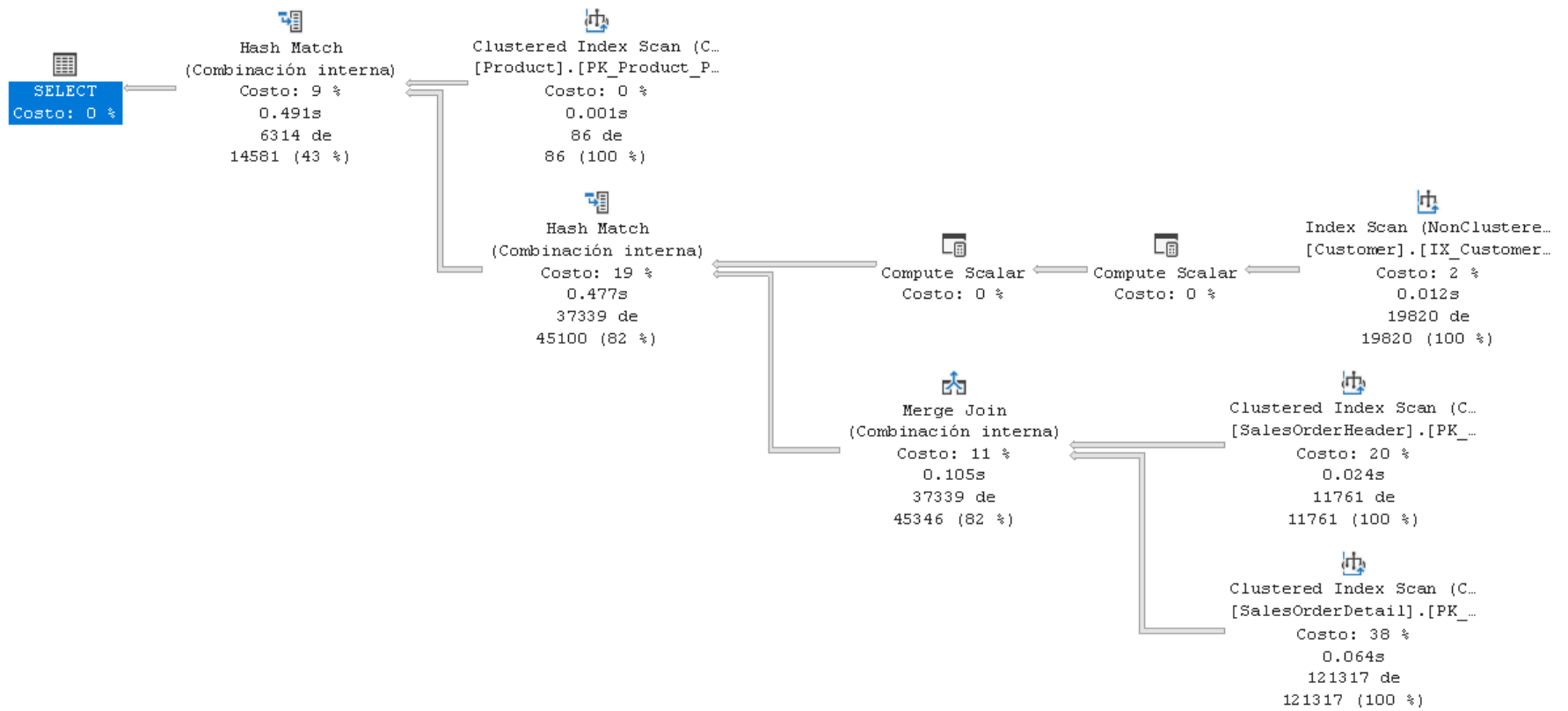
```

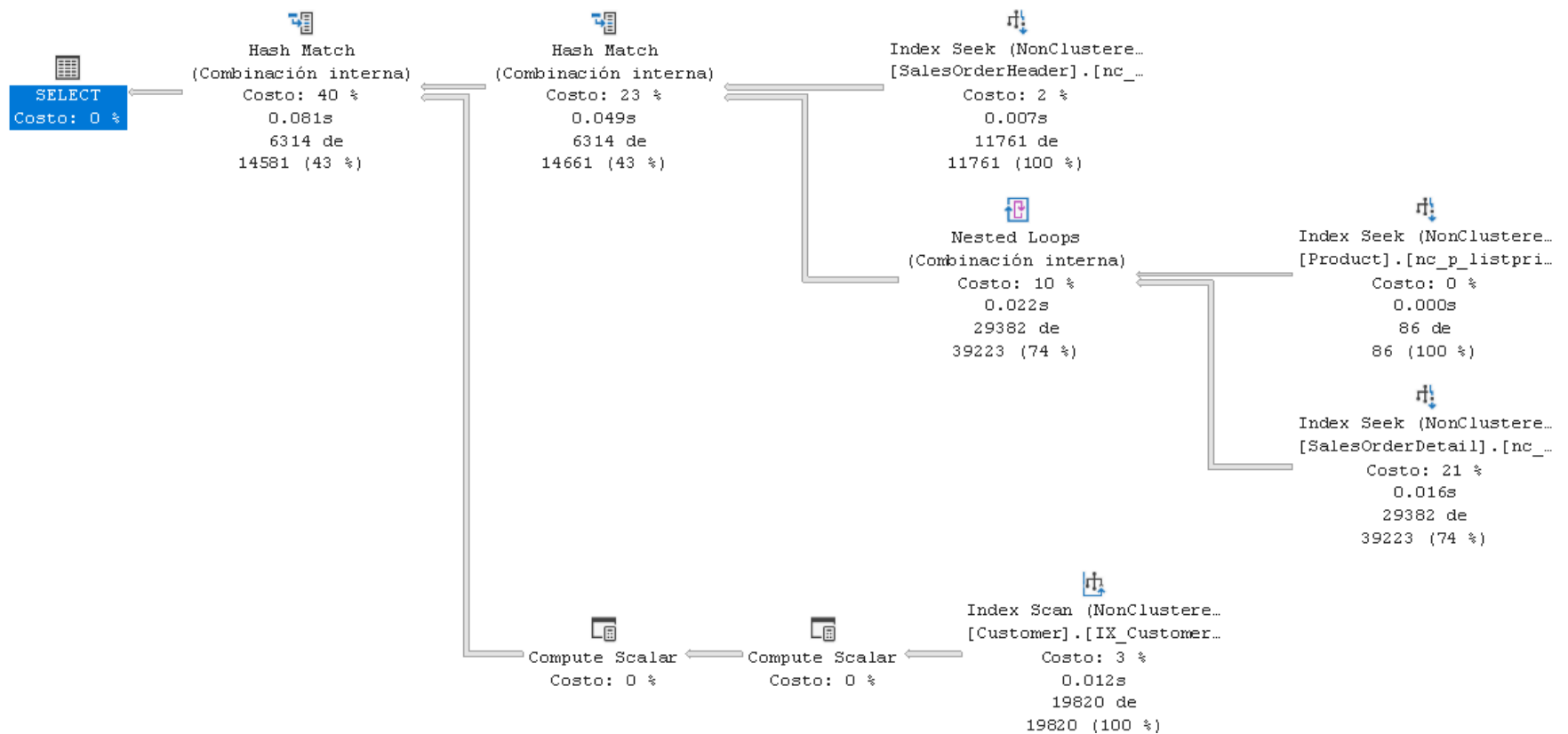
Creación de índices:

```
-- Creamos el índice optimizado
--creacion de indice nonclustered para Indice de fecha
CREATE NONCLUSTERED INDEX nc_soh_orderdate
ON Sales.SalesOrderHeader(OrderDate)
INCLUDE (CustomerID);
GO
--Ejecutamos de nuevo la consulta despues de crear los indices
-- Creamos el índice optimizado (2)
--creacion de indice nonclustered para Indice de precio
CREATE NONCLUSTERED INDEX nc_p_listprice
ON adventureworks2022.production.product(listprice)
include (name)
go
--creamos un nuevo indice para desaparecer el index seek sobre
salesorderdetail
CREATE NONCLUSTERED INDEX nc_sod_productid
on adventureworks2022.sales.salesorderdetail(productid)
include (orderqty)
drop index nc_sod_productid on
adventureworks2022.sales.salesorderdetail
go
```

Comparación de los planes de ejecución:

Base de datos Distribuidas – Práctica 2





Conclusión:

Primero fue de haber si esto funciona hasta ir viendo poco a poco que es lo mejor, es decir, ir entendiendo que aspectos se atacan desde que punto, si se debía crear un índice, si afectaban las lecturas.

Primero notamos el abrigo en la función del where y creímos que esto solucionaría gran parte de los Cluster index scan, pero realmente solo mejoro los tiempos, después note que habría que crear dos índices mas parecía fácil, solo crear los índices, pero resulto que crear índices no es solo crearlos si debíamos ver la lógica de que datos se solicitaban en la tabla y que índice primario se podía relacionar con los que tenia que crear.

Entonces ahora si se crearon los índices correctos con su respectivo include, lo cual si soluciono el problema real,

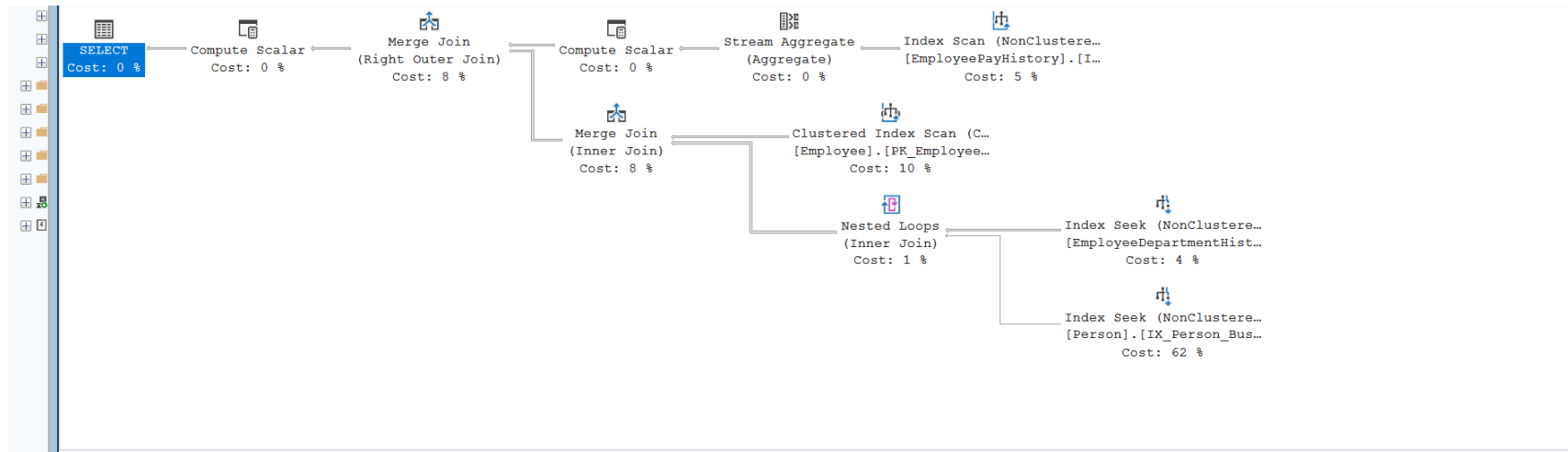
Base de datos Distribuidas – Práctica 2

ya que si no poníamos el include, al gestor le servían de nada porque decidía mejor usar index scan. Pero después de crear 3 índices correctos 1 para la fecha, otro para el precio de lista y otro para el id de la orden, con lo cual si se resolvió el tiempo, orden y uso de recursos del sistema para ejecutar esta consulta solo mediante los índices.

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
2	16-03-26	~25 ms	141 ms	Person=892 EDH=15 EPH=12 Employee=9	100% (BASE). Index Seek en Person representa el 62% del costo total del lote. Clustered Index Scan en Employee 10%	Subconsulta correlacionada en el SELECT. Falta de índices cubiertos provoca alto costo de I/O en la tabla Person al recuperar nombres y apellidos	Reestructurar consulta usando CTE. Crear índices cubiertos para Person, EmployeeDepartmentHistory y EmployeePayHistory

Plan de ejecución

Base de datos Distribuidas – Práctica 2



MENSAJES

(290 filas afectadas)

Table 'Person'.

Scan count 1, logical reads 892, physical reads 0, page server reads 0, read-ahead reads 0.

Table 'EmployeePayHistory'.

Scan count 1, logical reads 12, physical reads 0, page server reads 0, read-ahead reads 0.

Table 'Employee'.

Scan count 1, logical reads 9, physical reads 0, page server reads 0, read-ahead reads 0.

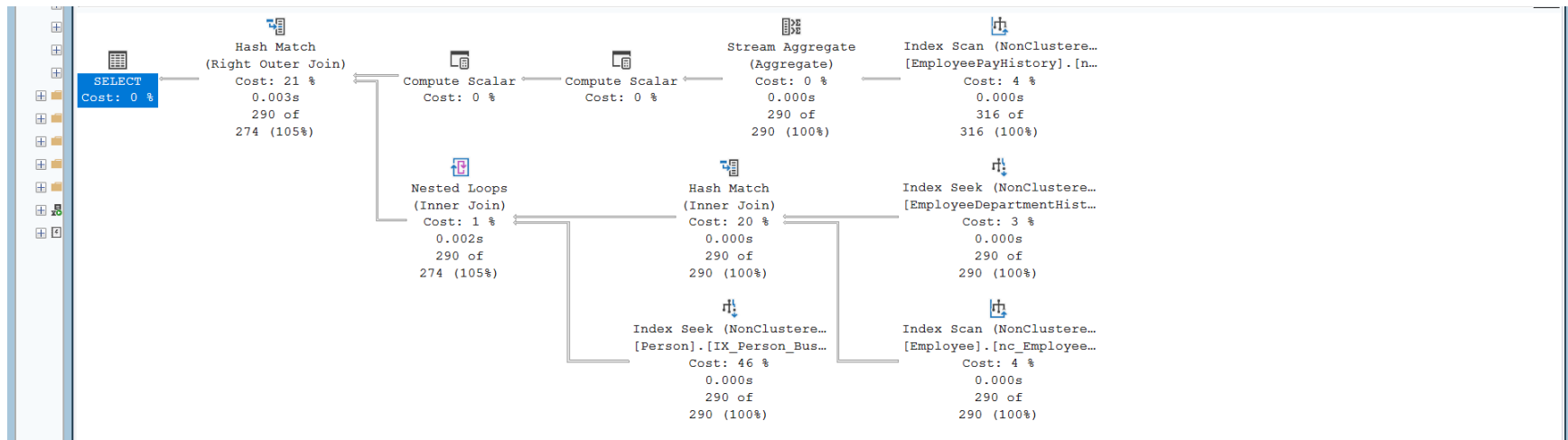
Table 'EmployeeDepartmentHistory'.

Scan count 1, logical reads 15, physical reads 0, page server reads 0, read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
CPU time = 15 ms, elapsed time = 25 ms.

Re-escribiendo la consulta:



Mensajes:
(290 filas afectadas)

Table 'Person'. Scan count 1, logical reads 4, physical reads 0.
Table 'EmployeePayHistory'. Scan count 1, logical reads 2, physical reads 0.
Table 'Employee'. Scan count 1, logical reads 2, physical reads 0.
Table 'EmployeeDepartmentHistory'. Scan count 1, logical reads 2, physical reads 0.

(1 fila afectada)

SQL Server Execution Times:

CPU time = 0 ms, elapsed time = 2 ms.

CONSULTAS:

<pre> SET STATISTICS IO ON; SET STATISTICS TIME ON; WITH PromedioSalarial AS (SELECT BusinessEntityID, AVG(Rate) AS PromedioSalario FROM HumanResources.EmployeePayHistory GROUP BY BusinessEntityID) SELECT e.NationalIDNumber, p.FirstName, p.LastName, edh.DepartmentID, ps.PromedioSalario FROM HumanResources.Employee e JOIN Person.Person p ON e.BusinessEntityID = p.BusinessEntityID JOIN HumanResources.EmployeeDepartmentHistory edh ON e.BusinessEntityID = edh.BusinessEntityID LEFT JOIN PromedioSalarial ps ON e.BusinessEntityID = ps.BusinessEntityID WHERE edh.EndDate IS NULL; SET STATISTICS IO OFF; SET STATISTICS TIME OFF; </pre>	<pre> with ProductsIds as(select p.ProductId, p.Name from Production.Product p JOIN Production.ProductSubcategory ps ON p.ProductSubcategoryID = ps.ProductSubcategoryID JOIN Production.ProductCategory pc ON ps.ProductCategoryID = pc.ProductCategoryID WHERE pc.ProductCategoryID IN (1, 2, 3) UNION select p.ProductId, p.Name from Production.Product p WHERE p.ListPrice > 500) SELECT sod.SalesOrderID, pi.ProductID, pi.Name FROM Sales.SalesOrderDetail sod inner join ProductsIds pi on sod.ProductID = pi.ProductId ; </pre>
--	---

Creación de índices:

```

-- 1. Índice para eliminar el costo del 62% en Person y evitar Key
Lookups
CREATE NONCLUSTERED INDEX nc_Person_BusinessEntityID_Cover
ON Person.Person (BusinessEntityID)

```

```
INCLUDE (FirstName, LastName);
GO

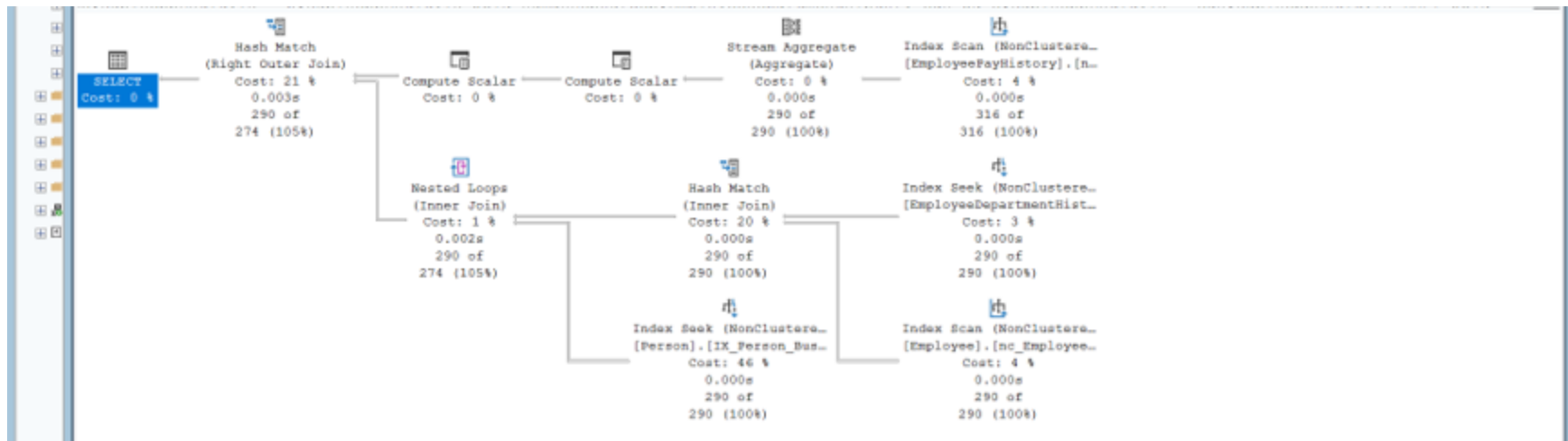
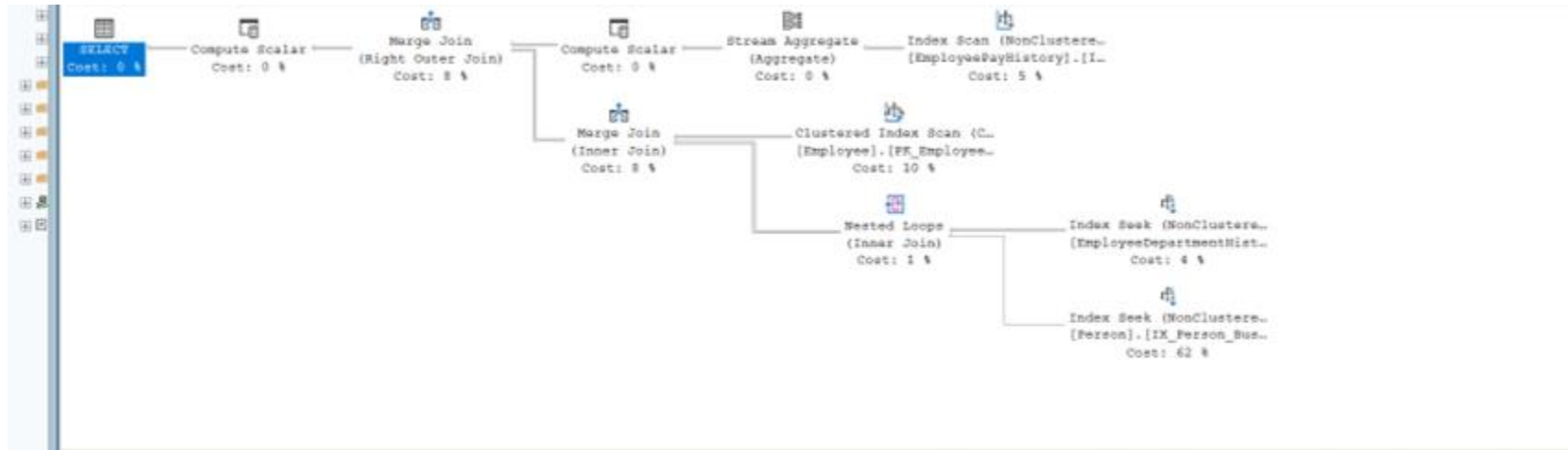
-- 2. Índice para filtrar instantáneamente a los empleados activos
CREATE NONCLUSTERED INDEX nc_EmpDeptHistory_EndDate
ON HumanResources.EmployeeDepartmentHistory (EndDate)
INCLUDE (BusinessEntityID, DepartmentID);
GO

-- 3. Índice para la pre-agrupación del salario en la CTE
CREATE NONCLUSTERED INDEX nc_EmpPayHistory_BusinessEntityID
ON HumanResources.EmployeePayHistory (BusinessEntityID)
INCLUDE (Rate);
GO

-- 4. Índice para cruce y selección en Employee
CREATE NONCLUSTERED INDEX nc_Employee_BusinessEntityID
ON HumanResources.Employee (BusinessEntityID)
INCLUDE (NationalIDNumber);
GO
```

Base de datos Distribuidas – Práctica 2

Comparación de los planes de ejecución:



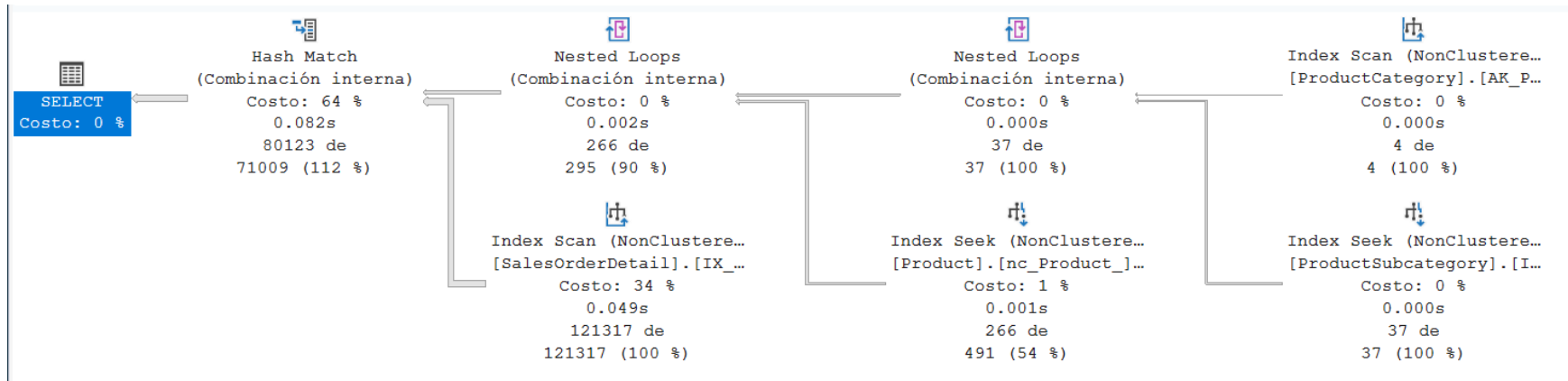
Conclusión:

Aunque el optimizador de SQL Server es lo suficientemente inteligente como para transformar la subconsulta original del SELECT en un Merge Join físico, el rendimiento se veía mermado por la falta de un diseño físico adecuado (índices). El costo relativo del 62% en la tabla Person evidenciaba que el motor estaba gastando la mayor parte de sus recursos buscando los campos FirstName y LastName. Al reescribir la consulta con una CTE para pre-agrupar los salarios y agregar índices cubiertos específicos, se logra que SQL Server resuelva toda la consulta directamente desde la memoria caché (reduciendo las lecturas lógicas a un solo dígito por tabla), logrando el límite de optimización relacional posible.

PLANES DE EJECUCIÓN

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
3	16-03-26	1006 ms.	141 ms	SOD= 273 Product=76	55%	Index scan de costo 33% Hash Match costo 63%	Se recomienda realizar un índice en subcategory ID y en categoryID

Base de datos Distribuidas – Práctica 2



MENSAJES

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, ...
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, ...
Table 'SalesOrderDetail'. Scan count 1, logical reads 273, physical reads 1, ...
Table 'Product'. Scan count 37, logical reads 76, physical reads 4...
Table 'ProductSubcategory'. Scan count 4, logical reads 8, physical reads 1, ...

SQL Server Execution Times:

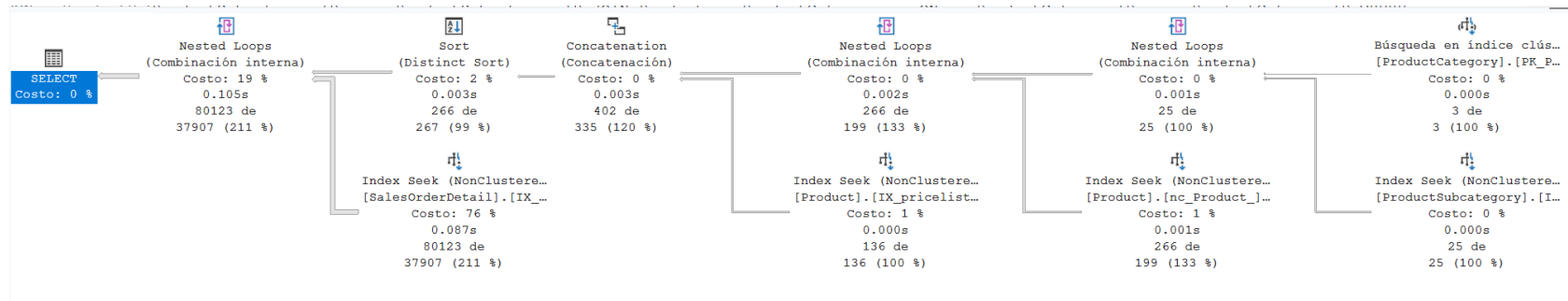
CPU time = 94 ms, elapsed time = 1012 ms.

SQL Server parse and compile time: e

CPU time = 0 ms, elapsed time = 0 ms

Re-escribiendo la consulta:

Base de datos Distribuidas – Práctica 2



Mensajes:

Table 'SalesOrderDetail'. Scan count 266, logical reads 908, physical reads 138, page server reads 0, ...
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
Table 'Product'. Scan count 26, logical reads 56, physical reads 5, page server reads 0, read-ahead reads 2, ...
Table 'ProductSubcategory'. Scan count 3, logical reads 6, physical reads 1, page server reads 0, ...
Table 'ProductCategory'. Scan count 3, logical reads 6, physical reads 1, page server reads 0, ...

SQL Server Execution Times:

CPU time = 78 ms, elapsed time = 783 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 1 ms.

CONSULTAS:

```
SELECT
sod.SalesOrderID,
p.ProductID,
p.Name
FROM Sales.SalesOrderDetail sod
JOIN Production.Product p
    ON sod.ProductID = p.ProductID
JOIN Production.ProductSubcategory ps
    ON p.ProductSubcategoryID =
ps.ProductSubcategoryID
JOIN Production.ProductCategory pc
    ON ps.ProductCategoryID =
pc.ProductCategoryID
WHERE
    pc.ProductCategoryID IN (1,2,3)
    OR p.ListPrice > 500;
```

```
with ProductsIds as(
    select
        p.ProductId,
        p.Name
    from Production.Product p
    JOIN Production.ProductSubcategory
ps
    ON p.ProductSubcategoryID =
ps.ProductSubcategoryID
    JOIN Production.ProductCategory pc
    ON ps.ProductCategoryID =
pc.ProductCategoryID
    WHERE
        pc.ProductCategoryID IN (1, 2, 3)
    UNION
    select
        p.ProductId,
        p.Name
    from Production.Product p
    WHERE p.ListPrice > 500
)
SELECT sod.SalesOrderID, pi.ProductID,
pi.Name
FROM Sales.SalesOrderDetail sod
inner join ProductsIds pi on
sod.ProductID = pi.ProductId ;
```

Creación de índices:

```
CREATE NONCLUSTERED INDEX ix_CSubctegoryid
ON production.productSubcategory (productcategoryid)
include (productsubcategoryid);
GO
```

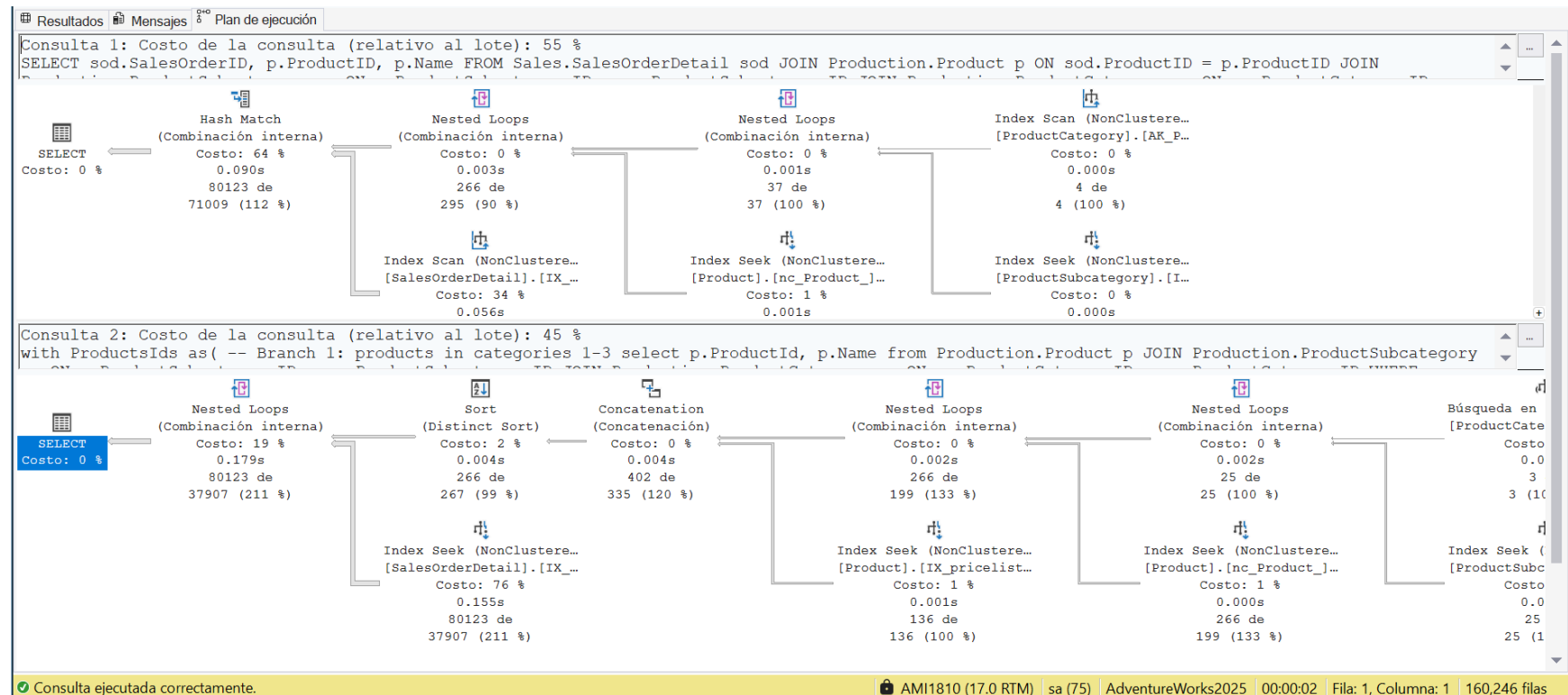
```
CREATE NONCLUSTERED INDEX ix_pricelist
ON production.product (listprice)
include (productid, name);

CREATE NONCLUSTERED INDEX nc1_SalesOLrderDetail_ProductID
ON Sales.SalesOrderDetail(ProductID)

-- indice Product con columnas necesarias
CREATE NONCLUSTERED INDEX nc_Product
ON Production.Product (ProductSubcategoryID)
INCLUDE (Name, listprice);

-- indice subcategory relacionar subcat y cat
CREATE NONCLUSTERED INDEX nc_Subcategory_CategoryID
ON Production.ProductSubcategory (ProductCategoryID)
INCLUDE (ProductSubcategoryID);
```

Comparación de los planes de ejecución:



Conclusión:

Aunque casi se triplicaron las lecturas lógicas de la tabla SalesOrderDetails, mejoro en tiempo de ejecución y en el costo total de la consulta. Como primera mejora, se pudo eliminar el Hash Match por un Nester Loop ya que se están obteniendo solo los valores filtrados para la visualización. Además, por la creación de los índices, podemos ver que se

están utilizando index seek en lugar de index scan lo cual hace que lea directamente las filas y reduce las operaciones de entrada y salida.

Ejercicio con la consulta 4:

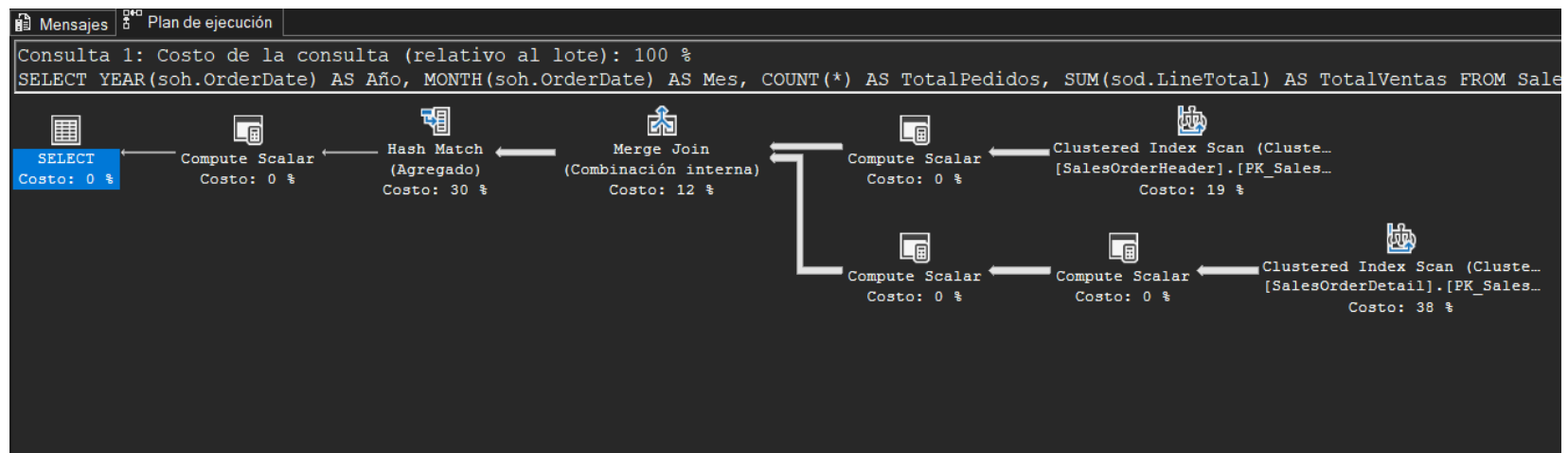
Consulta que genera un resumen de ventas mensual y anual que permite ver rápidamente qué meses fueron los más exitosos en términos de volumen de productos vendidos y dinero recaudado.

Query sin optimizar

Consulta que tiene problemas con los operadores COUNT(*), GROUP BY y JOIN

```
SELECT YEAR(soh.OrderDate) AS Año, MONTH(soh.OrderDate) AS Mes,  
       COUNT(*) AS TotalPedidos, SUM(sod.LineTotal) AS TotalVentas  
FROM Sales.SalesOrderHeader soh  
JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID  
GROUP BY YEAR(soh.OrderDate), MONTH(soh.OrderDate);
```

Plan de Ejecución:



Estadísticas de ejecución

```
SQL Server parse and compile time:
  CPU time = 47 ms, elapsed time = 125 ms.

SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.

(38 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server
Table 'SalesOrderDetail'. Scan count 1, logical reads 1241, physical reads 3, pa
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3, pag

SQL Server Execution Times:
  CPU time = 250 ms,  elapsed time = 331 ms.

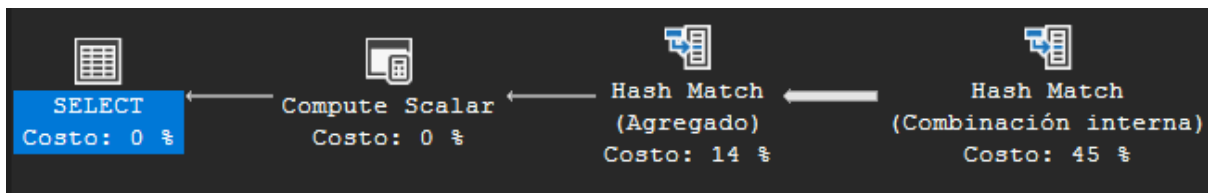
SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.

Hora de finalización: 2026-03-18T19:43:35.6132765-06:00
```

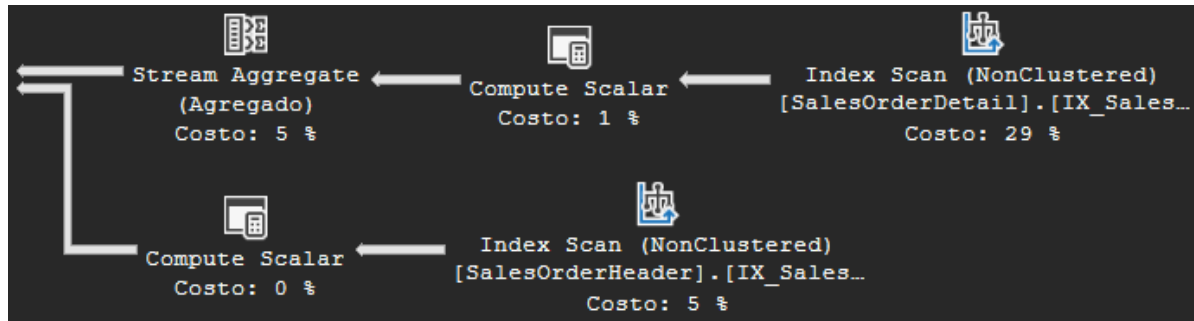
Query Optimizado:

```
SELECT
    soh.OrderYear AS Año,
    soh.OrderMonth AS Mes,
    COUNT(soh.SalesOrderID) AS TotalPedidos,
    SUM(Detalle.TotalPorOrden) AS TotalVentas
FROM Sales.SalesOrderHeader soh
JOIN (
    SELECT
        SalesOrderID,
        SUM(LineTotal) AS TotalPorOrden
    FROM Sales.SalesOrderDetail
    GROUP BY SalesOrderID
) Detalle ON soh.SalesOrderID = Detalle.SalesOrderID
GROUP BY
    soh.OrderYear,
    soh.OrderMonth;
```

Plan de ejecución:



Base de datos Distribuidas – Práctica 2



Estadísticas de ejecución

```
(38 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server
Table 'SalesOrderHeader'. Scan count 1, logical reads 73, physical reads 1, page server reads 0, read-ahead reads 71, pa
Table 'SalesOrderDetail'. Scan count 1, logical reads 467, physical reads 1, page server reads 0, read-ahead reads 465,

SQL Server Execution Times:
    CPU time = 156 ms,  elapsed time = 159 ms.

SQL Server Execution Times:
    CPU time = 0 ms,  elapsed time = 0 ms.

Hora de finalización: 2026-03-29T03:13:17.1862678-06:00
```

Índices Creados:

```
ALTER TABLE Sales.SalesOrderHeader ADD OrderYear AS YEAR(OrderDate) PERSISTED;
ALTER TABLE Sales.SalesOrderHeader ADD OrderMonth AS MONTH(OrderDate) PERSISTED;
GO
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_Agrupacion
ON Sales.SalesOrderHeader (OrderYear, OrderMonth)
INCLUDE (SalesOrderID);
GO
CREATE NONCLUSTERED INDEX IX_SalesOrderDetail_Skinny
ON Sales.SalesOrderDetail (SalesOrderID)
INCLUDE (LineTotal);
GO
```

Tabla comparativa:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan Costo(%)
4	18/03/2026	331	250	1927	100% (BASE)
4	18/03/2026	158	156	570	Reducido drásticamente

Issues Identificados	Recomendaciones	Consulta Optimizada
Clustered Index Scans completos. Hash	Implementar pre-agregación en tabla	No
Presencia de Hash Match por diferencia de ordenamiento en los índices (comportamiento esperado y óptimo)	Mantener estructura actual con índices "Skinny" y columnas persistidas. Representa el límite de optimización relacional	Si

Análisis:

La consulta no optimizada era funcional, sin embargo obligaba al motor de la base de datos a trabajar además. Al no tener índices, SQL Server leía la tabla de cabeceras (SalesOrderHeader) entera y la tabla de detalles (SalesOrderDetail) entera desde el disco físico hacia la memoria, lo que generó casi 2,000 lecturas lógicas. También generaba una tabla temporal en memoria de 120,000 filas cruzadas, a las que aplicaba las funciones de YEAR() y MONTH() fila por fila para extraer las fechas, lo que hacía que se destruyera la capacidad de usar índices.

Como los datos resultantes estaban desordenados, tenía que construir una estructura Hash en memoria para agrupar los años y meses y calcular las sumas. Por otro lado al usar COUNT(*), la consulta original estaba contando la cantidad de productos vendidos y no la cantidad de pedidos realizados en ese mes.

Dado estos problemas se buscó una manera de combinar dos técnicas, la reestructuración y diseño físico. Esta estrategia se basa en decirle a SQL Server que vaya directamente al índice miniatura (IX_SalesOrderDetail_Skinny) para leer solo los IDs y los totales y agruparlos inmediatamente mediante la subconsulta. Esto hace que las 120,000 filas se conviertan en 30,000 filas resumidas antes de hacer cualquier cruce.

Además, como pre-calcula OrderYear y OrderMonth físicamente en la tabla, el motor no tiene que usar la CPU para calcular fechas, lo que le permite leer los datos listos y ordenados. Esto deja que se cruce la tabla pre-agregada de detalles con la tabla de cabeceras, lo cual es muchísimo menos trabajo para la memoria y por último, al usar COUNT(soh.SalesOrderID), ahora sí contabiliza correctamente la cantidad de órdenes únicas realizadas en el mes.

Ejercicio con la consulta 6:

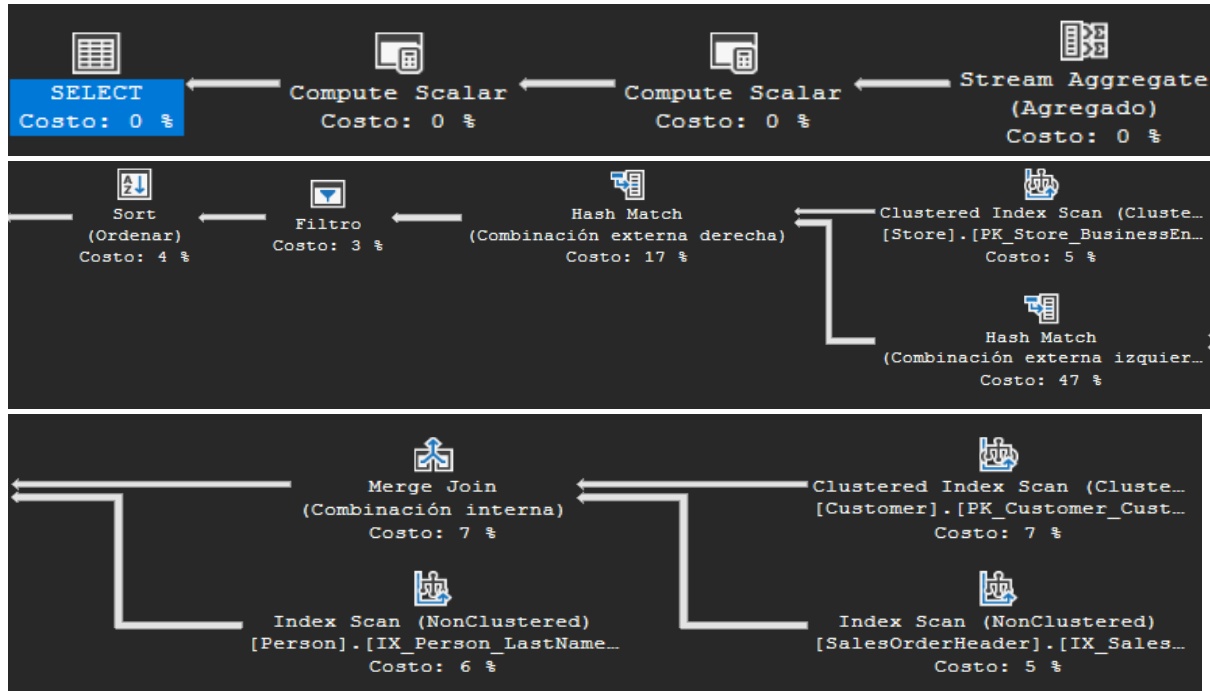
El objetivo de la consulta es obtener un listado de clientes cuyo nombre comienza con la letra "A" y contar cuántos pedidos ha realizado cada uno pero el problema es que no está optimizada

Query sin optimizar:

Consulta que tiene problemas al utilizar joins y group by

```
SELECT
    c.CustomerID,
    COALESCE(p.FirstName + ' ' + p.LastName, s.Name) AS Name,
    COUNT(soh.SalesOrderID) AS TotalPedidos
FROM Sales.Customer c
LEFT JOIN Person.Person p ON c.PersonID = p.BusinessEntityID
LEFT JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID
INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID
WHERE
    (p.FirstName LIKE 'A%' OR s.Name LIKE 'A%')
GROUP BY
    c.CustomerID,
    p.FirstName,
    p.LastName,
    s.Name;
```

Plan de ejecución:



Estadísticas

de

ejecución:

```
SQL Server parse and compile time:
  CPU time = 16 ms, elapsed time = 28 ms.

(1982 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server reads 0
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server reads 0
Table 'Person'. Scan count 1, logical reads 106, physical reads 1, page server reads 0, read-ahead reads 104, page server reads 0
Table 'SalesOrderHeader'. Scan count 1, logical reads 57, physical reads 1, page server reads 0, read-ahead reads 55, page server reads 0
Table 'Customer'. Scan count 1, logical reads 123, physical reads 1, page server reads 0, read-ahead reads 121, page server reads 0
Table 'Store'. Scan count 1, logical reads 103, physical reads 0, page server reads 0, read-ahead reads 0, page server reads 0

SQL Server Execution Times:
  CPU time = 109 ms,  elapsed time = 683 ms.

SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.

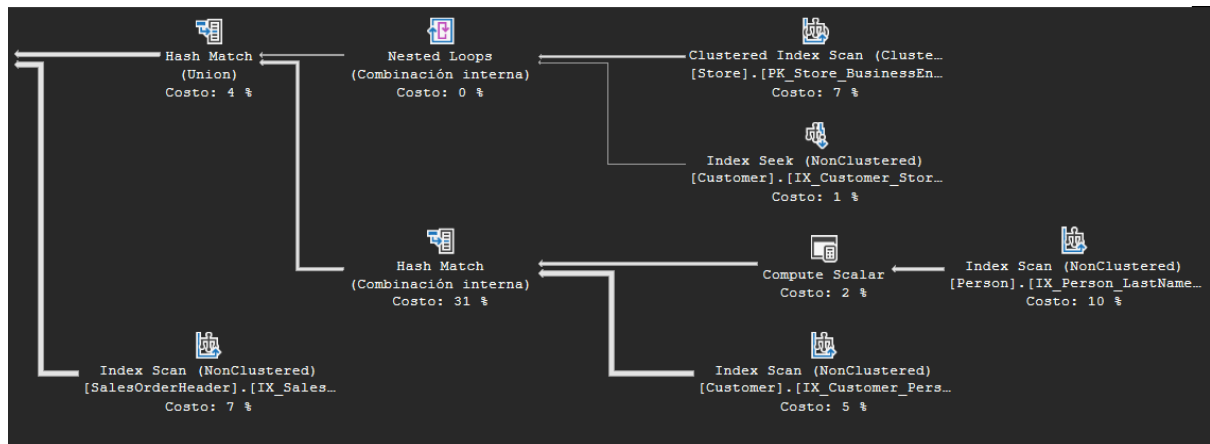
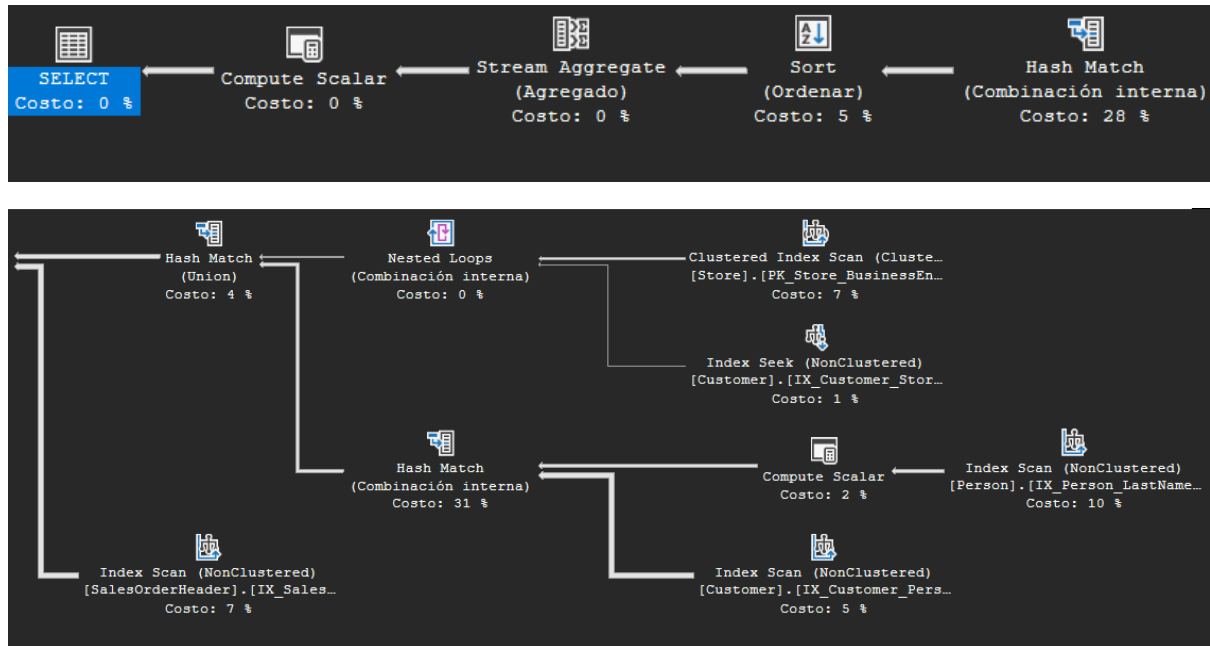
Hora de finalización: 2026-03-28T22:42:13.0887560-06:00
```

Query Optimizado:

Consulta que utiliza with y unión

```
WITH ClientesConA AS (  
    SELECT c.CustomerID, p.FirstName + ' ' + p.LastName AS Name  
    FROM Sales.Customer c  
    INNER JOIN Person.Person p ON c.PersonID = p.BusinessEntityID  
    WHERE p.FirstName LIKE 'A%'  
  
    UNION  
  
    SELECT c.CustomerID, s.Name AS Name  
    FROM Sales.Customer c  
    INNER JOIN Sales.Store s ON c.StoreID = s.BusinessEntityID  
    WHERE s.Name LIKE 'A%'  
)  
SELECT  
    c.CustomerID,  
    c.Name,  
    COUNT(soh.SalesOrderID) AS TotalPedidos  
FROM ClientesConA c  
INNER JOIN Sales.SalesOrderHeader soh ON c.CustomerID = soh.CustomerID  
GROUP BY  
    c.CustomerID,  
    c.Name;
```

Plan de ejecución:



Estadísticas de ejecución:

```
SQL Server parse and compile time:
  CPU time = 47 ms, elapsed time = 49 ms.

(1983 filas afectadas)
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page
Table 'SalesOrderHeader'. Scan count 1, logical reads 57, physical reads 0, page server reads 0, read-ahead reads
Table 'Customer'. Scan count 32, logical reads 120, physical reads 0, page server reads 0, read-ahead reads 12, p
Table 'Person'. Scan count 1, logical reads 106, physical reads 0, page server reads 0, read-ahead reads 0, page
Table 'Store'. Scan count 1, logical reads 103, physical reads 0, page server reads 0, read-ahead reads 0, page s

SQL Server Execution Times:
  CPU time = 47 ms,  elapsed time = 250 ms.
SQL Server parse and compile time:
  CPU time = 0 ms, elapsed time = 0 ms.

SQL Server Execution Times:
  CPU time = 0 ms,  elapsed time = 0 ms.

Hora de finalización: 2026-03-28T23:59:07.1472208-06:00
```

Índices Creados:

```
CREATE NONCLUSTERED INDEX IX_SalesOrderHeader_CustomerID_Optimized
ON Sales.SalesOrderHeader (CustomerID);
GO

CREATE NONCLUSTERED INDEX IX_Customer_PersonID
ON Sales.Customer (PersonID);
GO

CREATE NONCLUSTERED INDEX IX_Customer_StoreID
ON Sales.Customer (StoreID);
GO

CREATE NONCLUSTERED INDEX IX_Person_FirstName_Include_LastName
ON Person.Person (FirstName)
INCLUDE (LastName);
GO

CREATE NONCLUSTERED INDEX IX_Store_Name
ON Sales.Store (Name);
GO
```

Tabla comparativa:

Consulta ID	Fecha	Duración (ms)	CPU (ms)	Lecturas lógicas	Plan Costo(%)
6	28/03/2026	686	109	389	47% (Hash Match): Multiples Scans con costo
6	29/03/2026	371	94	191	38% (Hash Match): Operadores Seek ligeros

Issues Identificados	Recomendaciones	Consulta Optimizada
El uso de OR con LEFT JOIN confunde al optimizador e impide el uso eficiente de índices. Resulta en Clustered Index Scans masivos	Refactorizar la consulta usando la cláusula UNION para separar B2C y B2B. Crear índices cubiertos para acelerar las búsquedas y cruces	No
Ninguno. El plan ahora utiliza Index Seeks para filtrar rápidamente sin escanear tablas completas. I/O optimizando al máximo	Consulta óptima. Mantener los índices creados (Person, Store, SalesOrderHeader y los dos de Customer).	Si

Análisis:

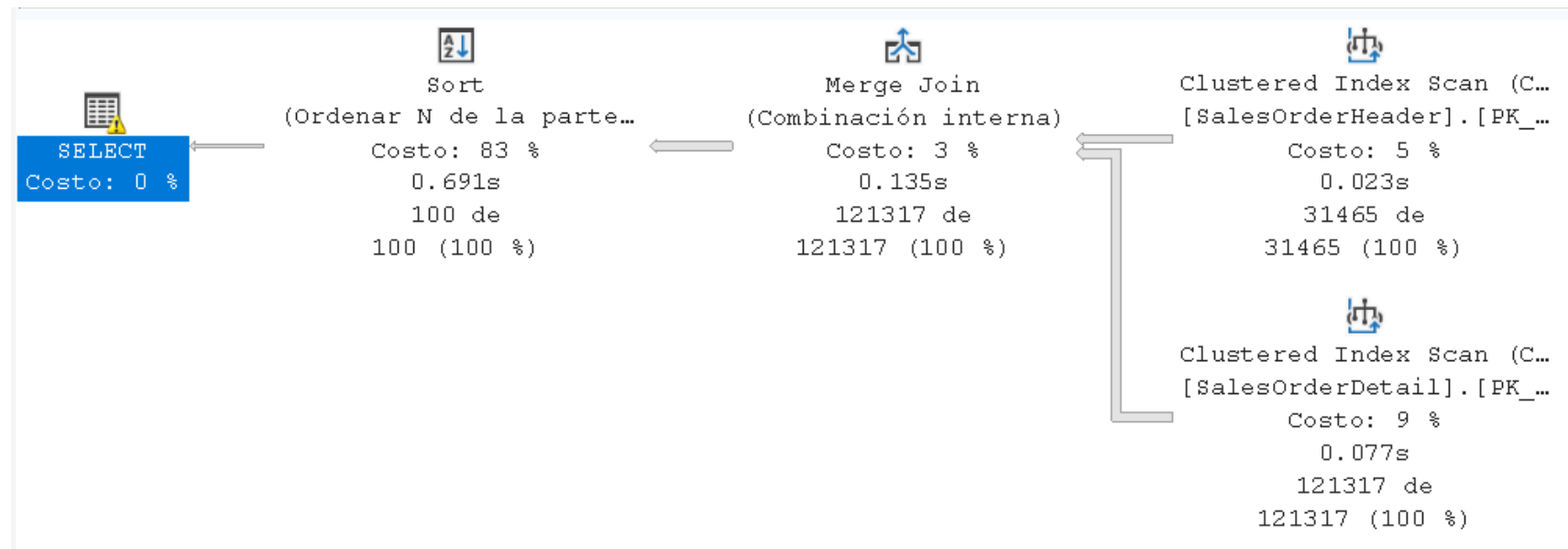
El problema principal de esta consulta radica en la sentencia where (p.FirstName LIKE 'A%' OR s.Name LIKE 'A%') ya que el optimizador no puede usar eficientemente un índice para buscar nombres con 'A' en Person y a la vez usar otro índice para buscar tiendas con 'A' en Store dentro del mismo árbol de ejecución, por lo que ignorará los índices, y decidirá hacer un **Scan completo** (leer todas las filas) tanto de la tabla Person . Person como de Sales . Store.

La solución es colocar índices cubiertos para transformar operaciones Scans en operaciones Seeks, o en otras palabras, pasar de directorios de clientes desde la 'A' a la 'Z', leyendo cada nombre y tienda para ver si empezaba con la 'A', a saltar directamente a la primera 'A', leer solo esa sección y detenerse inmediatamente en cuanto llega a la primera 'B'. Lo que hace que no se desperdicie tiempo ni memoria leyendo el resto del abecedario.

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
7	19-03-26	735 ms	641 ms	SOD=1238 SOH=686	Clustered index scan 9%	-Un sort -Un clustered INdex scan	-Crear indices necesario para tratar de desaparecer el sort o los clustered index

Plan de Ejecucion original

Base de datos Distribuidas – Práctica 2



Statistics de la consulta original

(100 filas afectadas)

Table 'SalesOrderDetail'.

Scan count 1,

logical reads 1238,

physical reads 0, page server reads 0,

read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderHeader'.

Scan count 1,

logical reads 686,

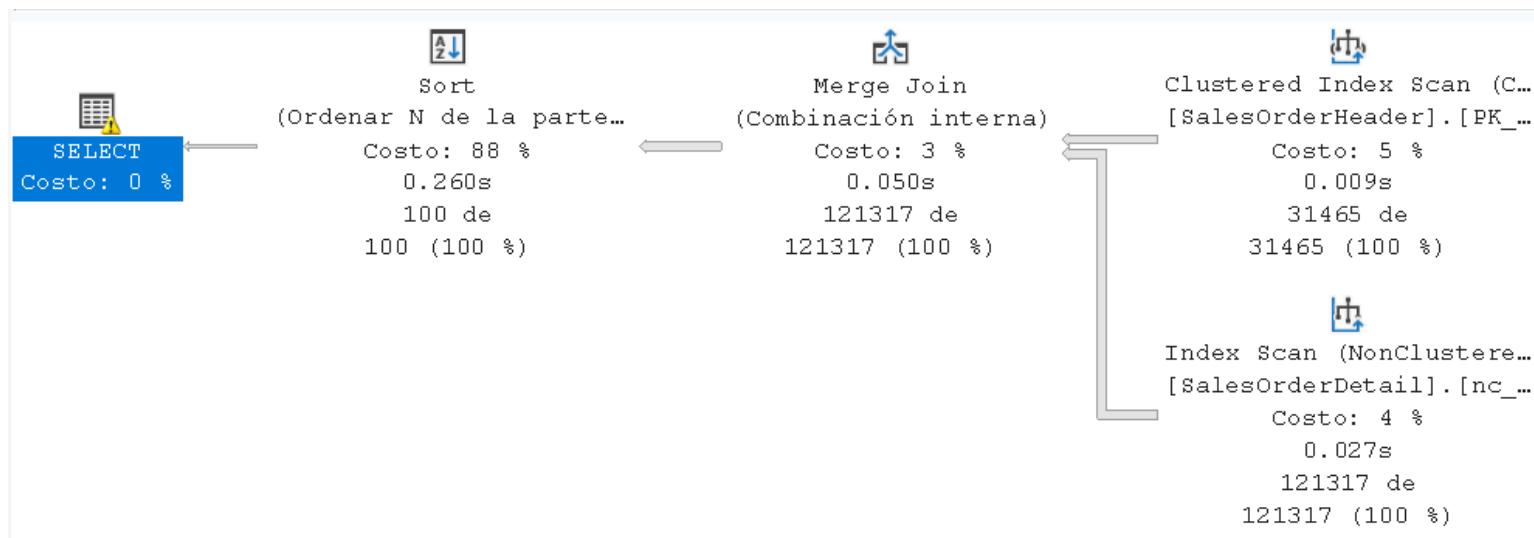
physical reads 0, page server reads 0,

read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
CPU time = 641 ms, elapsed time = 735 ms.

Plan de ejecución después de crear los índices



Con la creación de dos índices nuevos para que le sea mas fácil identificar lo que se consulta

Estadistics nuevos

(100 filas afectadas)
Table 'SalesOrderDetail'.
Scan count 1,
logical reads 362,
physical reads 0, page server reads 0,

Base de datos Distribuidas – Práctica 2

```
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderHeader'.
Scan count 1,
logical reads 686,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
    CPU time = 265 ms,  elapsed time = 315 ms.
```

Tabla comparativa:

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
7	19-03-26	735 ms	641 ms	SOD=1238 SOH=686	Clustered index scan 9%	-Un sort -Un clustered INdex scan	-Crear índices necesario para tratar de desaparecer el sort o los clustered index
7	23-03-26	315 ms	265 ms	SOD=362 SOH=686	Clustered index scan 5%	-Se lograron un index scan y mejorar los tiempos así como el uso de recursos	-Para esta consulta se logro hacerlo funcional y con bajo costo

La consulta para este caso no se modifico.

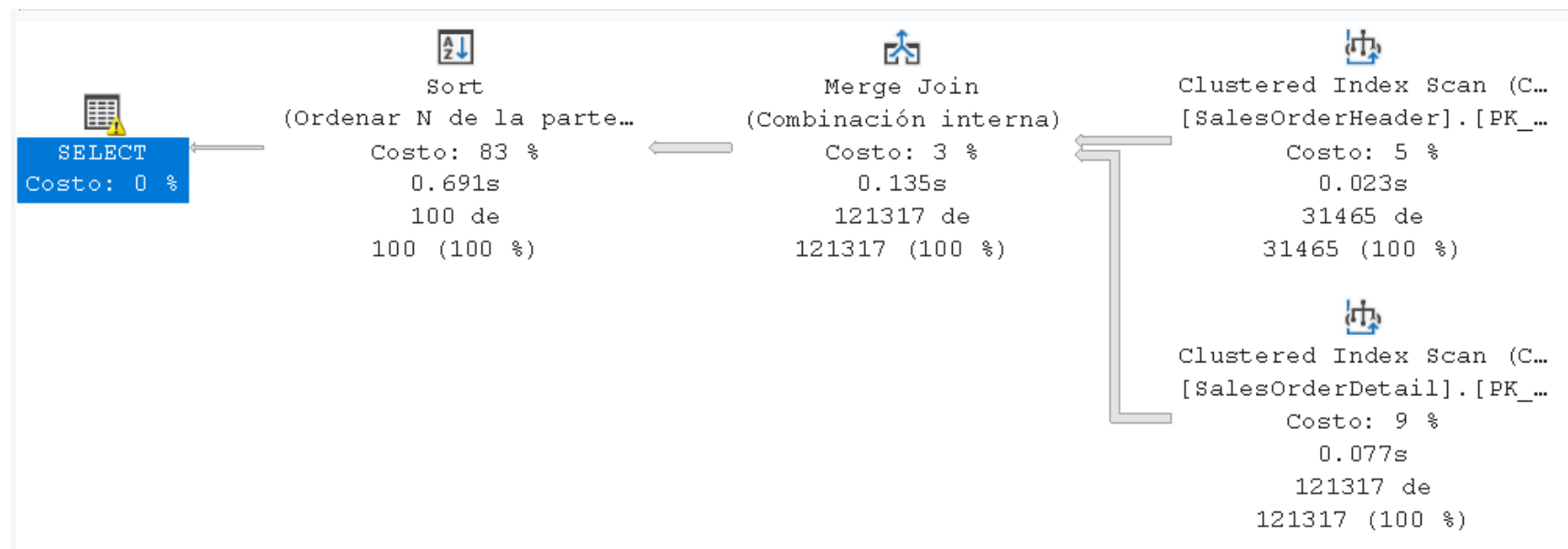
Creación de índices:

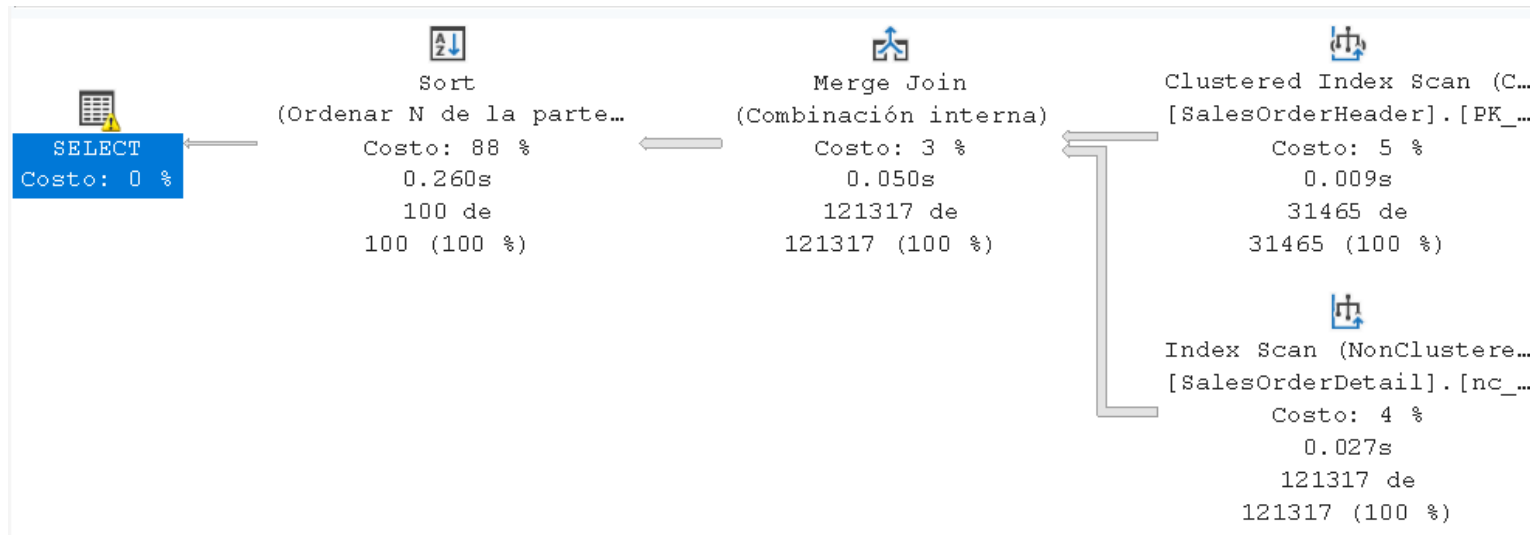
```
-- Índice 1: Para la cabecera (La fecha de envío y el puente)
```

```
CREATE NONCLUSTERED INDEX nc_soh_shipdate
ON Sales.SalesOrderHeader(SHIPDATE)
INCLUDE (OrderDate);
GO

-- Índice 2: Para el detalle (El puente y el contrabando)
CREATE NONCLUSTERED INDEX nc_sod_salesorderid
ON Sales.SalesOrderDetail(SalesOrderID)
INCLUDE (OrderQty, UnitPrice);
GO
```

Comparación de los planes de ejecución:



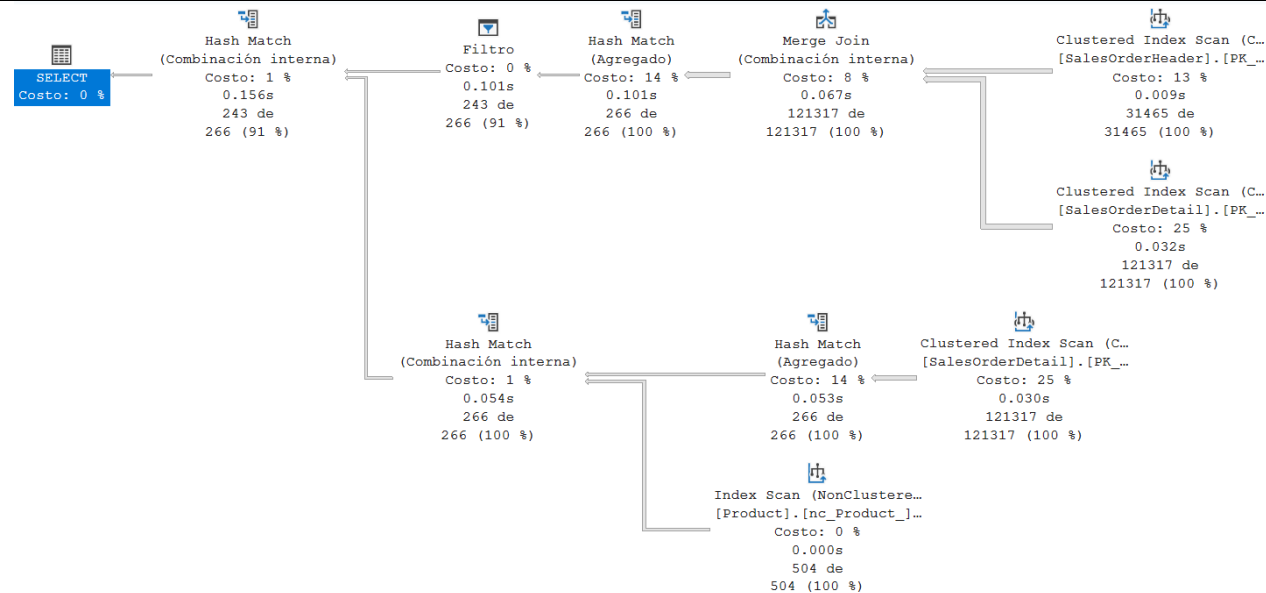


Conclusión:

Al analizar las estadísticas de la segunda consulta, se logró reducir las lecturas lógicas de SalesOrderDetail (de 1,238 a 362) mediante la creación de un Índice Cubierto sobre la llave foránea (SalesOrderID). Sin embargo, las lecturas de SalesOrderHeader se mantuvieron en 686 (un Clustered Index Scan completo). Esto buscando ayuda con la IA menciona que no es un error de indexación, sino una limitación matemática del motor: al exigir un ORDER BY con columnas de dos tablas distintas sin un filtro WHERE, el Optimizador se ve obligado a realizar el JOIN de la tabla completa antes de poder ejecutar el operador Sort para obtener el TOP 100, haciendo que el escaneo completo sea la vía de ejecución más eficiente.

Base de datos Distribuidas – Práctica 2

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
8	20- 03-26	365 ms	141 ms	Sod = 2482 Soh=686 P=6	62%	Clustered index scan Hash Match	Realizar una reescritura de la consulta e implementar índices en soh y sod



Mensajes:

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Product'. Scan count 1, logical reads 6, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'SalesOrderDetail'. Scan count 2, logical reads 2482, physical reads 0, page server reads 0, ...
 Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 0, page server reads 0, ...

SQL Server Execution Times:

CPU time = 141 ms, elapsed time = 365 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

Re-escritura de consulta:

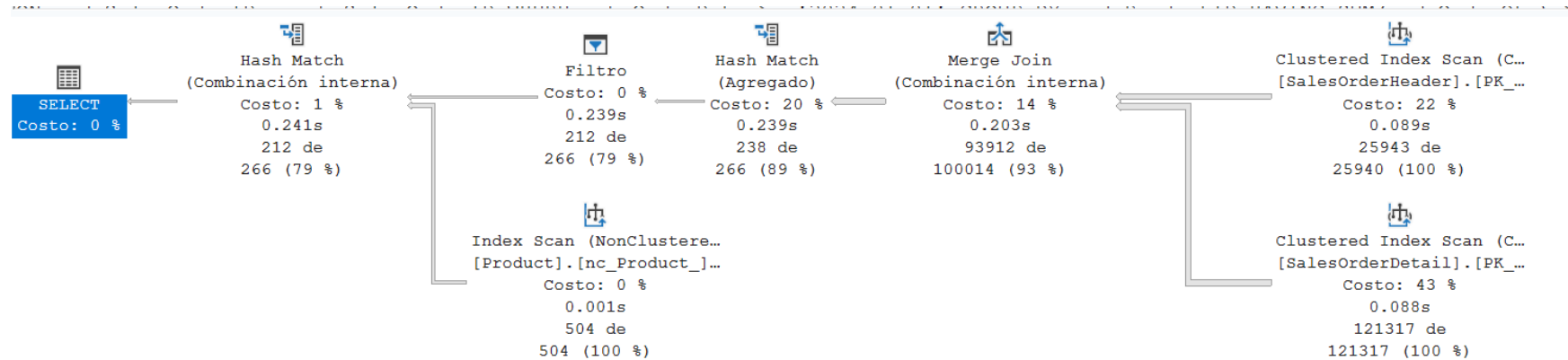


Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, ...
 Table 'Product'. Scan count 1, logical reads 6, physical reads 1, page server reads 0, read-ahead reads 4, ...
 Table 'SalesOrderDetail'. Scan count 1, logical reads 1241, physical reads 166, page server reads 0, read-ahead reads 1237, ...
 Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 378, page server reads 0, read-ahead reads 682, ...

SQL Server Execution Times:

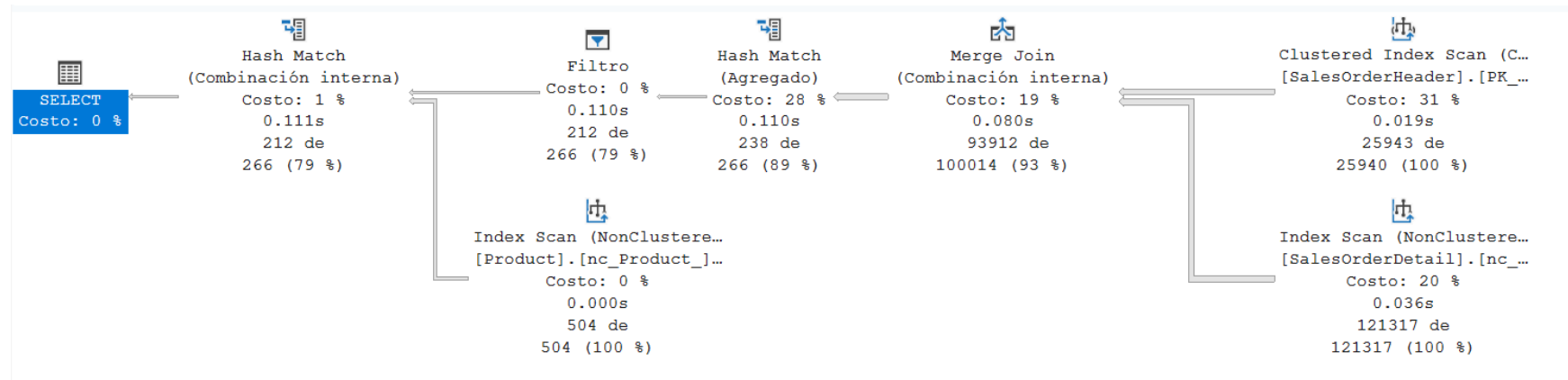
CPU time = 203 ms, elapsed time = 371 ms.

SQL Server parse and compile time:

CPU time = 0 ms, elapsed time = 0 ms.

Con creación de índices:

Base de datos Distribuidas – Práctica 2



```
Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0...
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0...
Table 'Product'. Scan count 1, logical reads 6, physical reads 0...
Table 'SalesOrderDetail'. Scan count 1, logical reads 303, physical reads 1...
Table 'SalesOrderHeader'. Scan count 1, logical reads 686, physical reads 3...
SQL Server Execution Times:
    CPU time = 94 ms,  elapsed time = 344 ms.
SQL Server parse and compile time:
    CPU time = 0 ms,  elapsed time = 0 ms.
```

Podemos notar que las lecturas lógicas de la tabla SalesOrderDetail bajaron a una cuarta parte, pero en la tabla SalesOrderHeader no hubo cambio en sus lecturas lógicas pero si en las lecturas físicas.

Consultas:

Original	Re-escritura
<pre> SELECT p.ProductID, p.Name, SUM(sod.OrderQty) AS TotalVendido FROM Production.Product p JOIN sales.SalesOrderDetail sod on sod.ProductID = p.ProductID WHERE p.ProductID IN (SELECT sod.ProductID FROM Sales.SalesOrderDetail sod JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID WHERE soh.OrderDate >= '2024- 01-01' GROUP BY sod.ProductID HAVING SUM(sod.OrderQty) > 100) GROUP BY p.ProductID, p.Name; </pre>	<pre> with DataSet as(SELECT sod.ProductID, sum(sod.OrderQty) as suma FROM Sales.SalesOrderDetail sod JOIN Sales.SalesOrderHeader soh ON sod.SalesOrderID = soh.SalesOrderID WHERE soh.OrderDate >= '2024- 01-01' GROUP BY sod.ProductID HAVING SUM(sod.OrderQty) > 100) SELECT p.ProductID, p.Name, ds.suma AS TotalVendido FROM Production.Product p inner join DataSet ds on p.ProductID = ds.ProductID </pre>

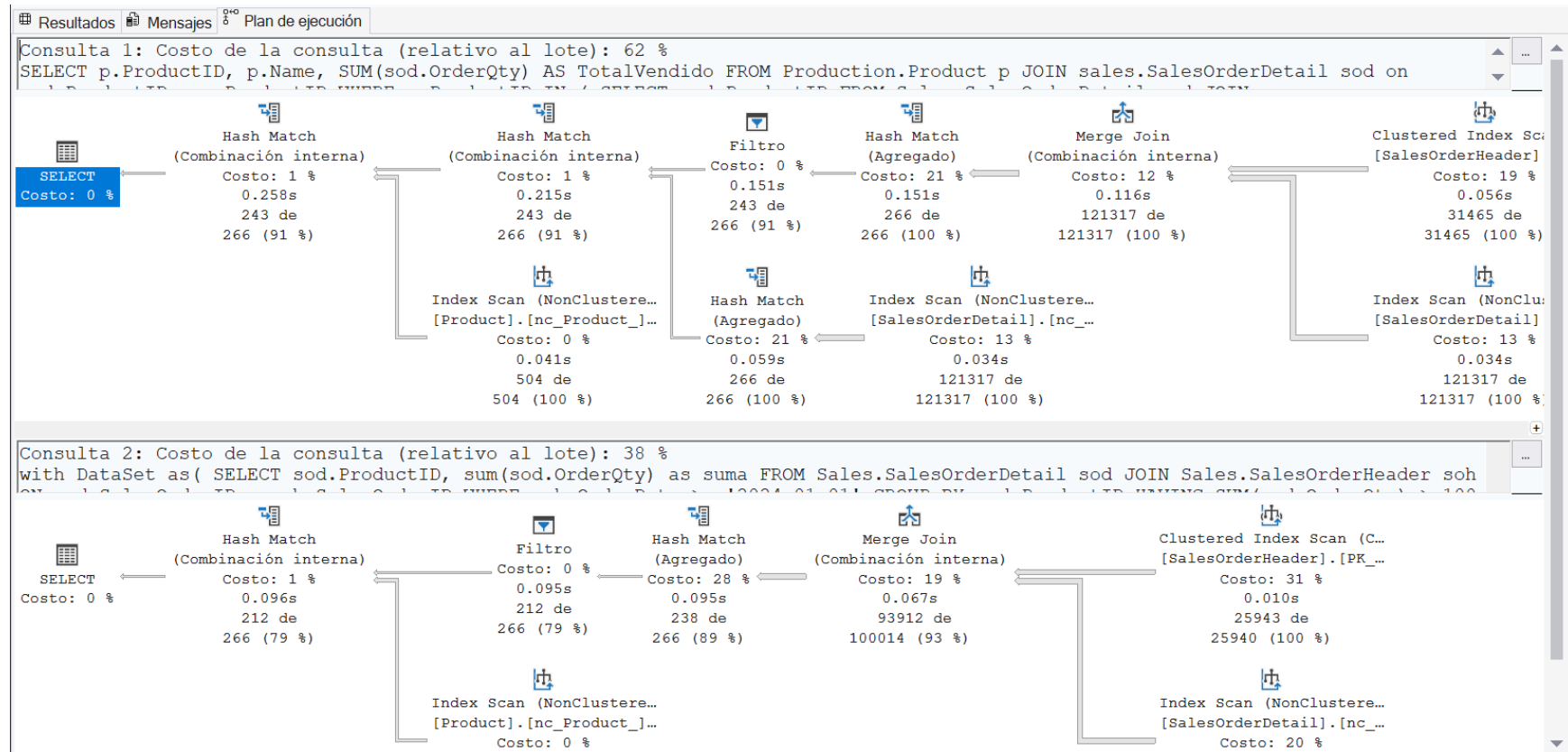
Índices

<pre> CREATE NONCLUSTERED INDEX nc_SalesOrderHeader_OrderDate ON Sales.SalesOrderHeader (OrderDate); CREATE NONCLUSTERED INDEX nc_SalesOrderDetail_OrderID_Cover ON Sales.SalesOrderDetail (SalesOrderID) INCLUDE (ProductID, OrderQty); </pre>
--

De igual manera que anteriores consultas, se implemento una CTE por la cantidad de operaciones JOIN's se están realizando, además de que primero filtra para después realizar la SUM en los datos necesarios.

Base de datos Distribuidas – Práctica 2

Comparación de plan de ejecución:



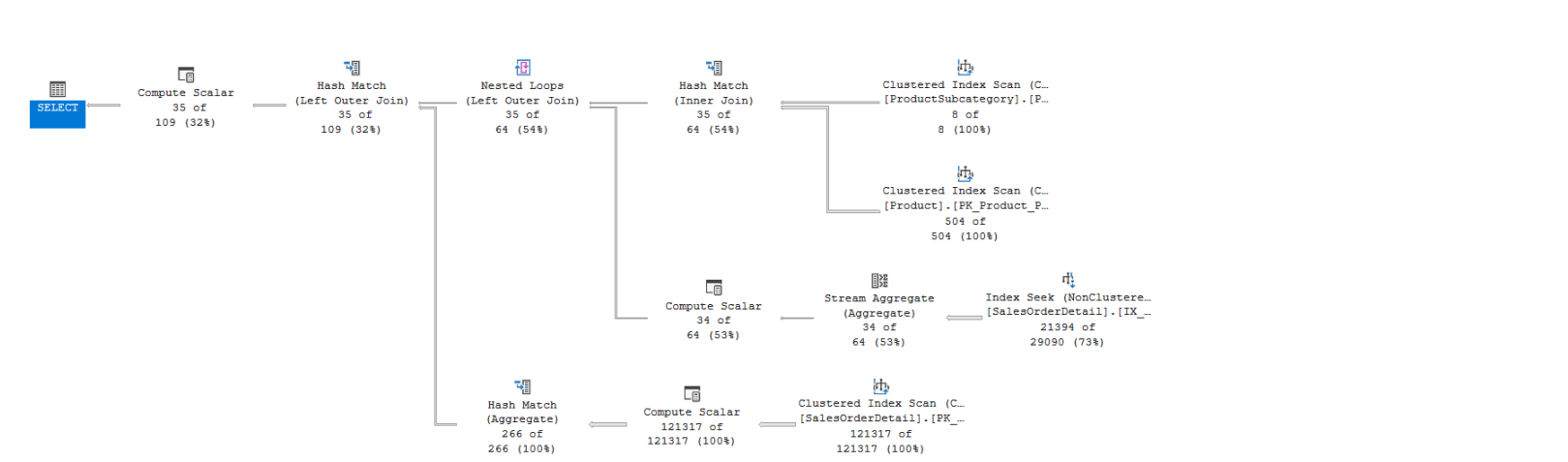
Conclusión

Al implementar una CTE se puede ver notoriamente el cambio aun antes de implementar los índices en donde no se hace una doble consulta a la tabla SalesOrderDetail. Al crear los índices con las columnas que ocuparemos, evitamos realizar demasiadas lecturas lógicas a la tabla SOD. -*

Base de datos Distribuidas – Práctica 2

CONS ULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	19-03-26	~1000 ms (00:00:01)	Alto (estimad o por Nested Loops)	Altas (múltiples lecturas por cada fila evaluada)	100% (BASE). Presencia de Nested Loops y Clustered Index Scans.	Subconsultas correlacionadas en el SELECT (RBAR). El motor ejecuta consultas repetitivas a SalesOrderDetail por cada producto. Se observan "Nested Loops" y escaneos completos.	Reestructurar la consulta extrayendo el SUM y COUNT hacia una Expresión de Tabla Común (CTE) o subconsulta en el FROM. Crear índices cubiertos.

Plan de ejecucion real ORIGINAL



Statics

Statistics originales
Statistics originales

(48 filas afectadas)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderDetail'.
Scan count 65,
logical reads 1368,
physical reads 0, page server reads 0,
read-ahead reads 805, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

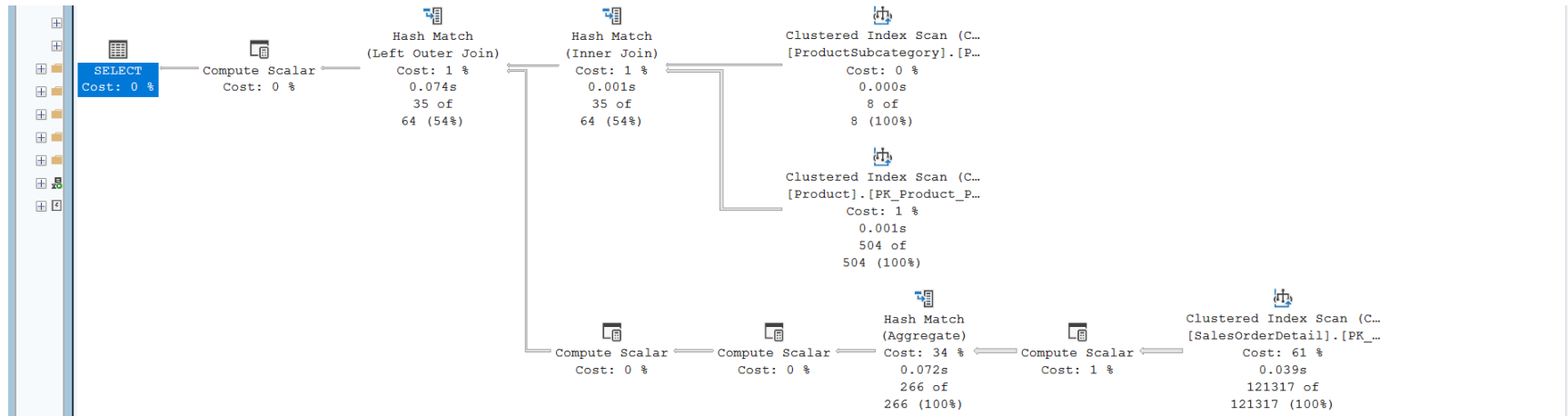
Table 'Product'.
Scan count 1,
logical reads 15,
physical reads 1, page server reads 0,
read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'ProductSubcategory'.
Scan count 1,
logical reads 2,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
CPU time = 984 ms, elapsed time = 1045 ms.

Plan Mejorado



Statistics finales

(35 filas afectadas)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderDetail'.
Scan count 1,
logical reads 1241,

physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Product'.

Scan count 1,

logical reads 15,

physical reads 0, page server reads 0,

read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'ProductSubcategory'.

Scan count 1,

logical reads 2,

physical reads 0, page server reads 0,

read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:

CPU time = 31 ms, elapsed time = 45 ms.

Tabla comparativa:

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
9	19-03-26	1045 ms	984 ms	SOD=1368 Product=15 ProductSubcat=2	Nested Loops (Left Outer Join) 54% Clustered Index Scan (Product) 100%	- Subconsultas correlacionadas en el SELECT generan efecto RBAR (Row-By-Agonizing-Row). - Escaneos repetitivos a la tabla SalesOrderDetail por cada producto encontrado.	- Reestructurar la consulta extrayendo las funciones de agregado hacia una Expresión de Tabla Común (CTE). - Implementar índices cubiertos en las llaves foráneas.
9	23-03-26	45 ms	31 ms	SOD=1241 Product=15 ProductSubcat=2	Hash Match (Aggregate) 34% Clustered Index Scan (SOD) 61%	- Se eliminaron los Nested Loops anidados. - El motor ahora realiza una lectura única y masiva sobre SalesOrderDetail, reduciendo drásticamente el consumo de CPU.	Mantener la estructura optimizada con la CTE. - Conservar el uso de LEFT JOIN con COALESCE para garantizar la exactitud de los datos sin sacrificar rendimiento.

CONSULTAS:

<pre> SET STATISTICS IO ON; SET STATISTICS TIME ON; SELECT p.ProductID, p.Name, (SELECT COUNT(*) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS VecesVendido, (SELECT SUM(sod.OrderQty * sod.UnitPrice) FROM Sales.SalesOrderDetail sod WHERE sod.ProductID = p.ProductID) AS Ingresos FROM Production.Product p JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID WHERE psc.ProductCategoryID = 3; SET STATISTICS IO OFF; SET STATISTICS TIME OFF; </pre>	<pre> -- Reescribiendo la consulta SET STATISTICS IO ON; SET STATISTICS TIME ON; WITH VentasProducto AS (SELECT ProductID, COUNT(SalesOrderID) AS VecesVendido, SUM(OrderQty * UnitPrice) AS Ingresos FROM Sales.SalesOrderDetail GROUP BY ProductID) SELECT p.ProductID, p.Name, COALESCE(vp.VecesVendido, 0) AS VecesVendido, COALESCE(vp.Ingresos, 0) AS Ingresos FROM Production.Product p JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID LEFT JOIN VentasProducto vp ON p.ProductID = vp.ProductID WHERE psc.ProductCategoryID = 3; SET STATISTICS IO OFF; SET STATISTICS TIME OFF; </pre>
---	---

Creación de índices:

```
-- Creando índices con include para facilitar el filtrado, cruce y la pre-
agrupación de la información
```

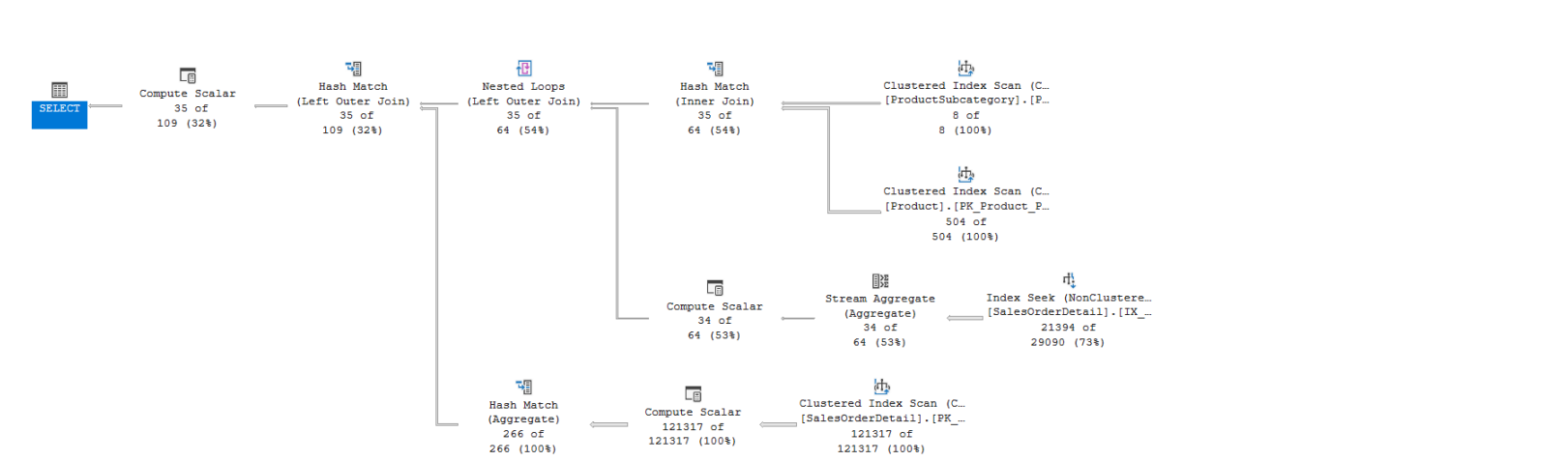
Base de datos Distribuidas – Práctica 2

```
CREATE NONCLUSTERED INDEX nc_sod_productid
ON Sales.SalesOrderDetail(ProductID)
INCLUDE (OrderQty, UnitPrice, SalesOrderID);
GO

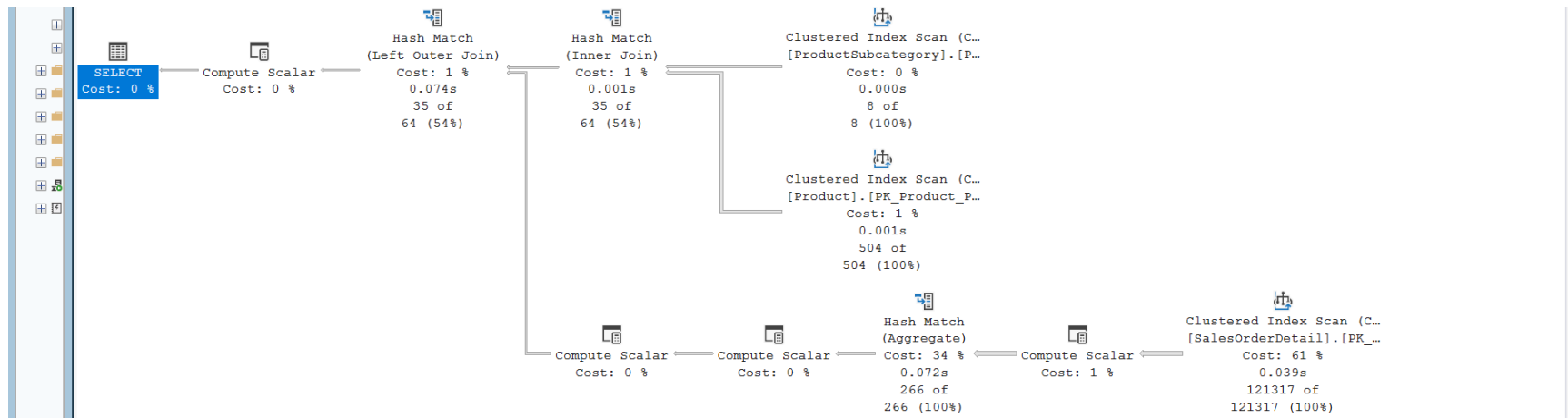
CREATE NONCLUSTERED INDEX nc_p_productsubcategoryid
ON Production.Product(ProductSubcategoryID)
INCLUDE (Name);
GO

CREATE NONCLUSTERED INDEX nc_psc_productcategoryid
ON Production.ProductSubcategory(ProductCategoryID)
INCLUDE (ProductSubcategoryID);
GO
```

Captura de los planes Comparados



Base de datos Distribuidas – Práctica 2



Conclusion:

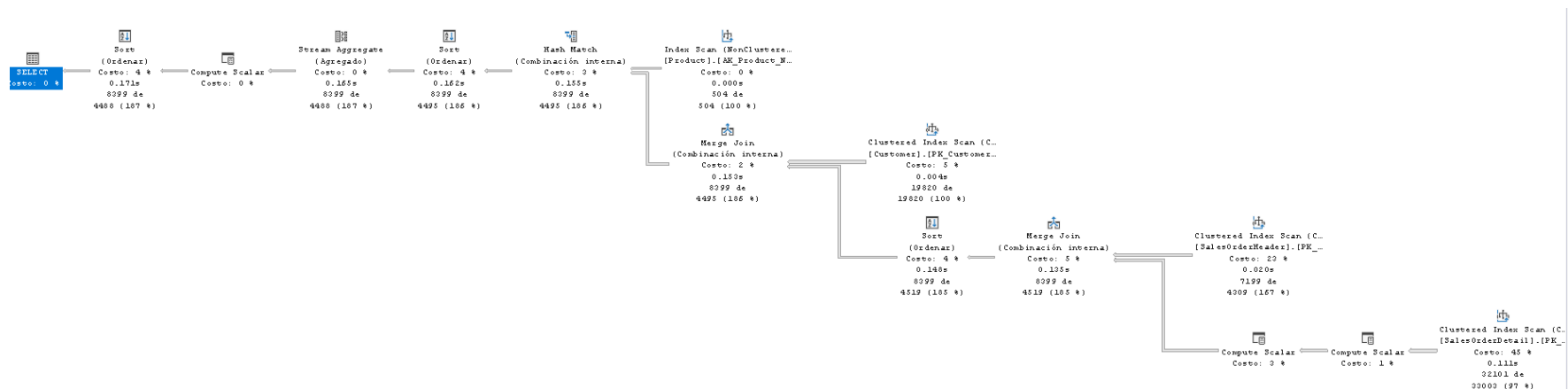
La optimización de la Consulta 9 evidencia el severo impacto negativo de utilizar subconsultas correlacionadas en la cláusula SELECT (antipatrón RBAR). Al refactorizar el código extrayendo los cálculos agregados (SUM y COUNT) hacia una Expresión de Tabla Común (CTE) y apoyando los cruces con la creación de índices no agrupados cubiertos, se transformó un proceso repetitivo y costoso en una lectura única y eficiente.

Este cambio de estrategia erradicó los pesados *Nested Loops* y *Clustered Index Scans*, reemplazándolos por *Index Seeks* y operadores *Hash Match* óptimos. Como resultado, se redujeron drásticamente las lecturas lógicas en la tabla de detalles y se desplomó el tiempo de ejecución (pasando de más de un segundo a tan solo unos milisegundos), demostrando que la combinación de una correcta escritura SQL (lógica basada en conjuntos) con un diseño físico adecuado (índices) es la clave para la escalabilidad y el rendimiento.

Base de datos Distribuidas – Práctica 2

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
10	19-03-26	441 ms	328 ms	SOD=1238 SOH=665 Customer=123 Product=6	Clustered index scan 45%	-3 clustered Index scan	-Crear indices necesario para tratar de desaparecer los clustered index

Plan de ejecucion real ORIGINAL



Statistics

Statistics originales
(8399 filas afectadas)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderDetail'.

Scan count 1,
logical reads 1238,
physical reads 0, page server reads 0,

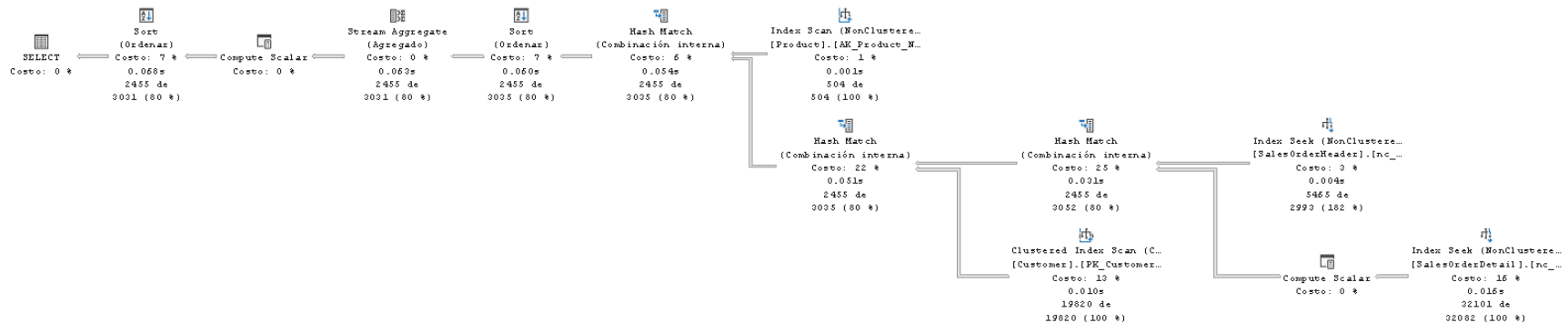
```
read-ahead reads 805, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'SalesOrderHeader'.
Scan count 1,
logical reads 665,
physical reads 0, page server reads 0,
read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Customer'.
Scan count 1,
logical reads 123,
physical reads 1, page server reads 0,
read-ahead reads 121, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.
Table 'Product'.
Scan count 1,
logical reads 6,
physical reads 1, page server reads 0,
read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server
reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
  CPU time = 328 ms,  elapsed time = 441 ms.
```

Plan Mejorado

Base de datos Distribuidas – Práctica 2



Statistics finales

(2455 filas afectadas)

Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Workfile'. Scan count 0, logical reads 0, physical reads 0, page server reads 0, read-ahead reads 0, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'Customer'.

Scan count 1,

logical reads 123,

physical reads 1, page server reads 0,

read-ahead reads 121, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderDetail'.

Scan count 1,

logical reads 152,

physical reads 3, page server reads 0,

read-ahead reads 148, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Table 'SalesOrderHeader'.

Scan count 1,

logical reads 23,

physical reads 1, page server reads 0,

read-ahead reads 21, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

Base de datos Distribuidas – Práctica 2

Table 'Product'.
 Scan count 1,
 logical reads 6,
 physical reads 1, page server reads 0,
 read-ahead reads 4, page server read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob page server reads 0, lob read-ahead reads 0, lob page server read-ahead reads 0.

(1 fila afectada)

SQL Server Execution Times:
 CPU time = 47 ms, elapsed time = 136 ms.

Tabla comparativa:

CONSULTA ID	Fecha	Duración (ms)	CPU (ms)	Lecturas Lógicas	Plan Costo (%)	Issues Identificados	Recomendaciones
10	19-03-26	441 ms	328 ms	SOD=1238 SOH=665 Customer=123 Product=6	Clustered index scan 45%	-3 clustered Index scan	-Crear indices necesario para tratar de desaparecer los clustered index
10	23-03-26	136 ms	47 ms	SOD=152 SOH=23 Customer=123 Product=6	Se cambio por un indeex seek		

CONSULTAS:

<pre> SET STATISTICS IO ON; SET STATISTICS TIME ON; SELECT c.PersonID AS Cliente, p.Name AS Producto, SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total, DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio </pre>	<pre> --Rescribiendo la consulta SET STATISTICS IO ON; SET STATISTICS TIME ON; SELECT c.PersonID AS Cliente, p.Name AS Producto, SUM(sod.OrderQty) AS Cantidad, SUM(sod.LineTotal) AS Total, </pre>
--	--

<pre> FROM Sales.SalesOrderHeader soh JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID JOIN Production.Product p ON sod.ProductID = p.ProductID WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5 AND DATEPART(quarter, soh.OrderDate) = 2 -- soh.OrderDate >= '20140401' AND soh.OrderDate < '20140701' AND sod.LineTotal > 1000 GROUP BY c.PersonID, p.Name, soh.OrderDate, soh.ShipDate ORDER BY Total DESC; SET STATISTICS IO Off; SET STATISTICS TIME Off; </pre>	<pre> DATEDIFF(day, soh.OrderDate, soh.ShipDate) AS DiasEnvio FROM Sales.SalesOrderHeader soh JOIN Sales.Customer c ON soh.CustomerID = c.CustomerID JOIN Sales.SalesOrderDetail sod ON soh.SalesOrderID = sod.SalesOrderID JOIN Production.Product p ON sod.ProductID = p.ProductID WHERE DATEDIFF(day, soh.OrderDate, soh.ShipDate) > 5 AND soh.OrderDate >= '20140401' AND soh.OrderDate < '20140701' AND sod.LineTotal > 1000 GROUP BY c.PersonID, p.Name, soh.OrderDate, soh.ShipDate ORDER BY Total DESC; SET STATISTICS IO Off; SET STATISTICS TIME Off; </pre>
--	--

Creación de índices:

```

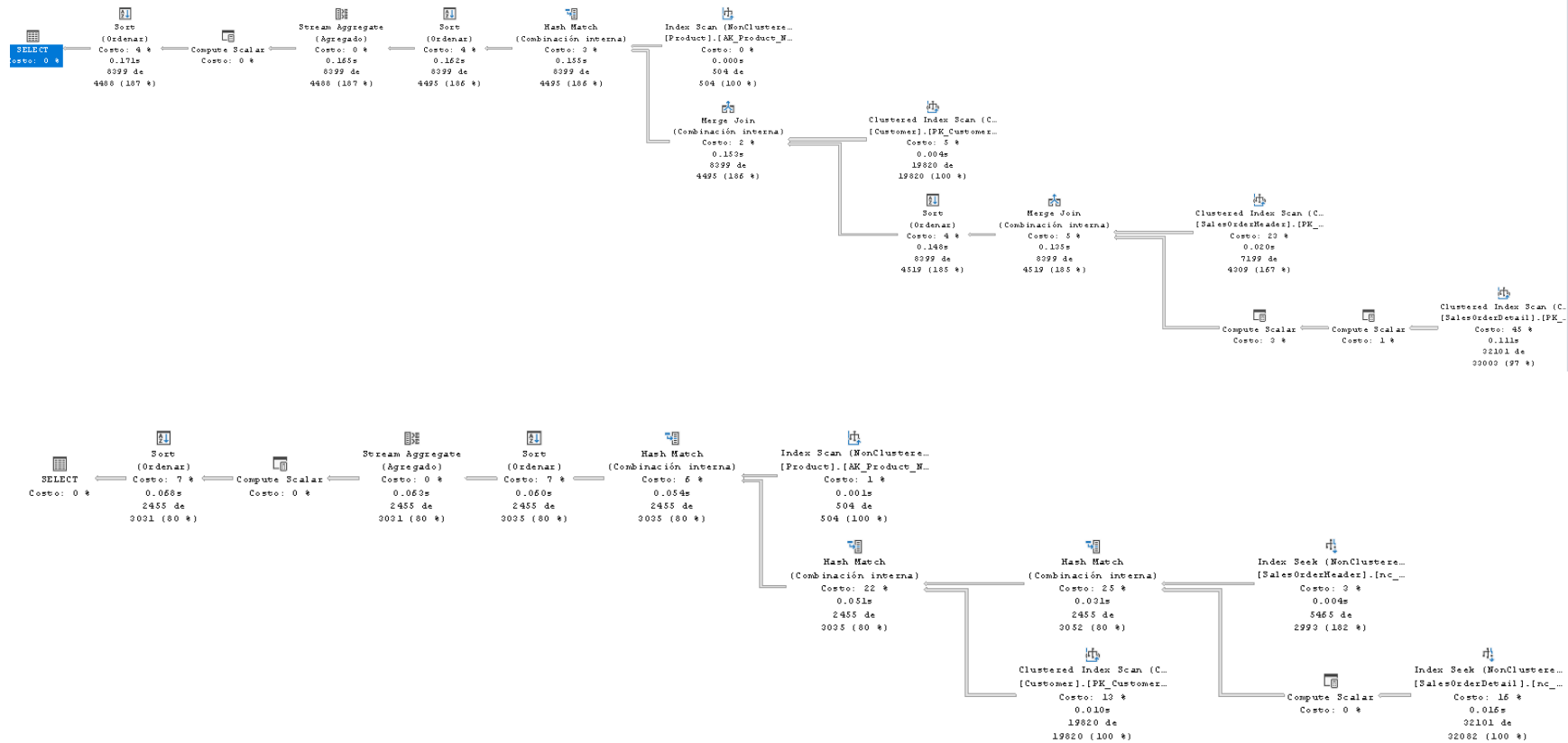
--Creando indices con include para facilitar el filtro de la
informacion
create nonclustered index nc_soh_orderdates
on sales.salesorderheader(orderdate)
include (shipdate, customerid)

create nonclustered index nc_sod_linetotal
on sales.salesorderdetail(linetotal)
include (orderqty, productid)

```

Captura de los planes Comparados

Base de datos Distribuidas – Práctica 2



Conclusión:

Analizando la consulta original vemos un problema muy fuerte de rendimiento ocasionado por el uso de la función `DATEPART()` en el `WHERE`. Esta práctica violaba el principio de *no ponerle abrigo a las funciones*, obligando al motor de base de datos a ignorar los índices y realizar un escaneo completo (Scan) para evaluar la función fila por fila en toda la tabla histórica.

Para resolver este cuello de botella. Primero, se modificó el código en la parte del `WHERE` reemplazando la función por un rango de fechas con operadores lógicos. Segundo, se diseñó e implementó un Índice No Agrupado Cubierto

(nc_soh_orderdates) definiendo la columna de filtrado (OrderDate) como llave principal e incluyendo las columnas requeridas por el SELECT y los JOIN para evitar el efecto de *Key Lookup que termina ignorando el indices por no contener las columnas necesarias vaya*.

Los resultados obtenidos a través de las STATISTICS IO y TIME confirmaron la mejoría de la consulta propuesta. SQL Server logró ejecutar un *Index Seek*, lo que redujo el impacto en la memoria RAM a tan solo 23 lecturas lógicas para la tabla principal SalesOrderHeader. También el tiempo de procesamiento (CPU Time) cayó a 47 ms, y el consumo de recursos temporales en disco (Workfiles y Worktables) se mantuvo en cero, validando que el cruce de las cuatro tablas, sumado a GROUP BY y ORDER BY, se resolvió completa la forma más escalable y óptima posible.